Cómo se construyó este backend

Guía educativa, de lo genérico a lo específico
FastAPI + SQLAlchemy (async) + Alembic + PostgreSQL

Sistema de punto de venta — arquitectura por capas
Documento de estudio, generado a partir del código real del proyecto

Índice

1. Introducción: cómo usar esta guía

2. Fundamentos de Python que vas a necesitar

3. Las piezas del stack: FastAPI, Pydantic, SQLAlchemy, Alembic

4. Arquitectura general: diseño por capas

5. El patrón CRUD simple, explicado una sola vez con Category

6. Infraestructura core, a fondo

7. Caso de estudio: Product (la primera validación cruzada)

8. Caso de estudio: Stock

9. Caso de estudio a fondo: Sale (el flujo más complejo)

10. Purchase y Exchange: variaciones del mismo patrón

11. Caso de estudio a fondo: Promotion + PromotionItem y el algoritmo de combos

12. Testing: cómo se prueba todo esto

13. Migraciones con Alembic

14. Convenciones y buenas prácticas: checklist

15. Próximos pasos sugeridos (para que practiques vos)

1. Introducción: cómo usar esta guía

Este documento explica, de principio a fin, cómo está construido el backend de un sistema de punto de venta (POS). No es sólo un resumen de "qué hace" cada archivo: el objetivo es que entiendas **por qué** el código está escrito así, para que puedas reproducirlo vos mismo, extenderlo, o corregir un bug sin copiar y pegar sin entender lo que estás pegando.

La guía está organizada de lo genérico a lo específico:

- Primero, los fundamentos de Python que el proyecto usa mucho y quizás no manejes con soltura todavía (type hints modernos, `dataclasses`, `async`/`await`, clases genéricas, interfaces).
- Después, una introducción a las herramientas del stack (FastAPI, Pydantic, SQLAlchemy, Alembic) y por qué se eligió cada una.
- Luego la arquitectura general: cómo se organiza el proyecto en capas y por qué.
- Y recién ahí entramos al código real, entidad por entidad, del caso más simple (una tabla de catálogo como `Category`) al más complejo (el algoritmo de combos de `Promotion`).

Importante: el proyecto tiene seis entidades de catálogo simple (Category, Brand, Color, Size, Branch, PaymentMethod) que son *estructuralmente idénticas* entre sí — el mismo patrón repetido seis veces con distintos nombres de campo. Por eso esta guía explica ese patrón **una sola vez**, usando `Category` como ejemplo canónico, y después asume que ya lo entendiste cuando aparezcan las otras cinco.

> **Cómo recomiendo leer esto:** la primera vez, de corrido, en orden. Después, usalo como referencia: cuando tengas que tocar algo parecido a lo que ya viste acá, volvé a la sección correspondiente en vez de adivinar.

2. Fundamentos de Python que vas a necesitar

Antes de mirar el código del proyecto, repasemos algunas construcciones de Python moderno que vas a encontrar en **todos** los archivos. Si ya las conocés, salteá esta sección; si no, es la base para entender todo lo que sigue.

2.1. Type hints modernos: `int | None`

Python no obliga a declarar tipos, pero permite *anotarlos* para que herramientas (editores, linters, otros programadores) sepan qué esperar. Desde Python 3.10, en vez de escribir `Optional[int]` (la forma vieja, que requiere `from typing import Optional`), se puede escribir `int | None` directamente:

```
def buscar_producto(id_producto: int) -> str | None:
   # devuelve el nombre del producto, o None si no existe
   ...
```

Esto se lee: "recibe un `int`, devuelve un `str` o `None`". En el proyecto vas a ver esto constantemente en los campos opcionales de las clases, por ejemplo `color_id: int | None = None` significa "puede tener un id de color, o puede no tener ninguno".

2.2. `@dataclass`: clases de datos sin escribir `__init__`

Si quisieras representar un producto con una clase común de Python, tendrías que escribir esto:

```
class Producto:
   def __init__(self, nombre, precio):
       self.nombre = nombre
       self.precio = precio

   def __repr__(self):
       return f"Producto(nombre={self.nombre!r}, precio={self.precio!r})"

   def __eq__(self, other):
       return self.nombre == other.nombre and self.precio == other.precio
```

El decorador `@dataclass` genera automáticamente el `__init__`, el `__repr__` (para que se imprima bien en consola/debug) y el `__eq__` (para poder comparar dos instancias con `==`). Lo mismo de arriba, con `dataclass`:

```
from dataclasses import dataclass

@dataclass
class Producto:
   nombre: str
   precio: float
```

El proyecto usa además `@dataclass(slots=True)`. El parámetro `slots=True` le dice a Python que no permita agregar atributos nuevos a las instancias después de creadas, y a cambio la clase ocupa menos memoria y es un poco más rápida de acceder. Para las clases de dominio del proyecto (que son objetos simples, sólo datos, sin lógica adicional) esto es una buena práctica: no necesitás la flexibilidad de agregar atributos al vuelo, así que ganás performance gratis.

> **Por qué el proyecto no usa Pydantic para las clases de dominio:** Pydantic también genera clases de datos, pero además valida tipos en tiempo de ejecución y sabe serializar a JSON. Eso está buenísimo para la capa que habla con el mundo exterior (la API), pero el dominio (las reglas de negocio puras) no debería depender de una librería de validación HTTP. Por eso el dominio usa `dataclass` (pura biblioteca estándar de Python) y Pydantic aparece recién en `app/schemas/`. Vas a ver esta separación una y otra vez.

2.3. `async` / `await`: concurrencia sin hilos

Cuando tu programa hace una consulta a una base de datos, la mayor parte del tiempo no está "pensando" — está esperando que la base de datos responda por la red. Durante esa espera, un programa síncrono (normal) se queda bloqueado sin hacer nada. Un programa **asíncrono** puede aprovechar esa espera para atender otra petición mientras tanto.

En Python, una función se declara asíncrona con `async def`, y dentro de ella usás `await` para "ceder el control" mientras esperás algo (una consulta a la base, una petición HTTP, etc.):

```
async def obtener_producto(id_producto: int) -> Producto | None:
   resultado = await session.execute(select(ProductModel).where(ProductModel.id ==
id_producto))
   return resultado.scalar_one_or_none()
```

La palabra clave `await` sólo se puede usar dentro de una función `async def`, y para **llamar** a una función `async` siempre tenés que poner `await` adelante (si no, en vez de ejecutarse te devuelve un objeto "corrutina" sin ejecutar, un error común al empezar). En este proyecto, prácticamente todo lo que toca la base de datos es `async`: los repositorios, los servicios, las rutas. FastAPI y SQLAlchemy (en su modo async) están diseñados para trabajar así de punta a punta.

> **No hace falta que entiendas el event loop en detalle** para seguir esta guía. Lo importante para leer el código es: (1) toda función que dice `async def` se llama con `await`, y (2) eso permite que el servidor atienda muchas peticiones en simultáneo sin usar un hilo del sistema operativo por cada una.

2.4. Clases abstractas e interfaces: `ABC` y `@abstractmethod`

Una *interfaz* es un contrato: define qué métodos debe tener una clase, sin decir cómo los implementa. En Python esto se hace heredando de `ABC` (Abstract Base Class) y marcando los métodos con `@abstractmethod`:

```
from abc import ABC, abstractmethod

class CategoryRepository(ABC):
   @abstractmethod
   async def get_by_id(self, category_id: int) -> "Category | None": ...

class SqlAlchemyCategoryRepository(CategoryRepository):
   async def get_by_id(self, category_id: int) -> "Category | None":
       # implementación real con SQLAlchemy
       ...
```

Si intentaras instanciar `CategoryRepository()` directamente, Python te lo impide, porque tiene métodos abstractos sin implementar. La idea es: el resto del código (los servicios) depende de la *interfaz* `CategoryRepository`, no de la implementación concreta `SqlAlchemyCategoryRepository`. Esto se llama **inversión de dependencias**: si mañana cambiás de PostgreSQL a otra base, o querés testear sin base

de datos real, sólo tenés que escribir otra clase que cumpla la misma interfaz — el resto del código no se entera.

2.5. Genéricos: `TypeVar` y `Generic[T]`

A veces querés escribir código que funcione con *cualquier* tipo, pero mantenga la relación entre los tipos de entrada y salida. Por ejemplo: "una lista de productos" y "una lista de categorías" deberían compartir el mismo código de "listar todo", sin mezclar productos con categorías por error. Para eso Python tiene genéricos:

```
from typing import TypeVar, Generic

T = TypeVar("T")

class Caja(Generic[T]):
   def __init__(self, contenido: T) -> None:
       self.contenido = contenido

caja_de_productos: Caja[Producto] = Caja(Producto("Remera", 20000))
caja_de_categorias: Caja[Categoria] = Caja(Categoria("Ropa"))
```

`T` es una variable de tipo: un "comodín" que representa "el tipo que sea, pero siempre el mismo dentro de esta instancia". El proyecto usa esto en `SqlAlchemyRepository[ModelT]` y `CrudService[DomainT]` (las clases base genéricas, ver sección 6) para escribir la lógica de "listar/crear/actualizar/borrar" **una sola vez** y reutilizarla para las seis entidades de catálogo simple, sin perder el chequeo de tipos.

2.6. `Protocol`: interfaces "por estructura"

`ABC` (sección 2.4) exige herencia explícita: una clase tiene que heredar de la interfaz para "contar" como esa interfaz. `Protocol` es distinto: cualquier clase que tenga los métodos con la firma correcta cuenta como esa interfaz, **sin heredar de nada**. Es "si camina como pato y grazna como pato, es un pato":

```
from typing import Protocol

class TieneNombre(Protocol):
   nombre: str

def saludar(algo: TieneNombre) -> str:
   return f"Hola, {algo.nombre}"

# Cualquier clase con un atributo .nombre sirve acá, sin heredar de TieneNombre
```

El proyecto usa esto en `CrudRepositoryProtocol` (sección 6.4): el servicio genérico `CrudService` no necesita que los repositorios hereden de una clase común, sólo que tengan los métodos `list_all`/`get_by_id`/`create`/`update`/`delete` con la firma esperada.

2.7. Decoradores y `@staticmethod`

Un decorador (la sintaxis `@algo` arriba de una función o clase) es una forma de "envolver" esa función o clase con comportamiento extra, sin modificar su código. Ya viste `@dataclass` y `@abstractmethod`. Otro que vas a ver mucho es `@staticmethod`: marca un método que no necesita acceso a `self` (a la instancia), sólo agrupa lógica relacionada dentro de la clase por prolijidad:

```
class SqlAlchemyCategoryRepository:
   @staticmethod
   def _to_domain(model: "CategoryModel") -> "Category":
       return Category(id=model.id, name=model.name, description=model.description)
```

Se llama como `SqlAlchemyCategoryRepository._to_domain(modelo)` o `self._to_domain(modelo)` — no hace falta instanciar nada para usarlo, pero vive adentro de la clase porque conceptualmente le pertenece a ella.

3. Las piezas del stack

Con los fundamentos de Python cubiertos, veamos qué hace cada librería grande que usa el proyecto, antes de ver cómo se combinan.

3.1. FastAPI: el framework web

FastAPI es un framework para construir APIs HTTP en Python. Sus dos ideas centrales para este proyecto son:

- **Rutas declaradas con decoradores**: `@router.get("/categories")` asocia una URL y un método HTTP a una función Python.

Inyección de dependencias con `Depends`: en vez de que cada función de ruta construya "a mano" sus dependencias (una sesión de base de datos, un servicio), las declara como parámetros con `Depends(...)`, y FastAPI se encarga de resolverlas antes de llamar a la función. Esto lo vas a ver constantemente:

```
@router.get("/categories/{category_id}")
async def get_category(
   category_id: int, service: CategoryService = Depends(get_category_service)
) -> Category:
   return await service.get_by_id(category_id)
```

- Acá `category_id` viene de la URL (FastAPI lo extrae automáticamente porque coincide con `{category_id}` en el path), y `service` lo arma la función `get_category_service` (que a su vez depende de una sesión de base de datos, ver sección 6.2) cada vez que llega una petición.

3.2. Pydantic: validación y serialización

Pydantic define modelos de datos que **validan** automáticamente: si el JSON que llega no tiene la forma esperada (falta un campo, un número viene como texto no convertible, etc.), Pydantic rechaza la petición con un error 422 antes de que tu código de negocio la vea. Además sabe convertir esos modelos hacia y desde JSON. En el proyecto, cada entidad tiene sus propios modelos Pydantic en `app/schemas/` — nunca el dominio ni la base de datos, sólo la capa que habla HTTP.

3.3. SQLAlchemy: el ORM

Un ORM (Object-Relational Mapper) te deja trabajar con la base de datos usando clases y objetos de Python en vez de escribir SQL a mano. SQLAlchemy define *modelos mapeados*: clases que representan tablas, donde cada atributo es una columna:

```
class CategoryModel(Base):
   __tablename__ = "categories"

   id: Mapped[int] = mapped_column(primary_key=True)
   name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
```

`Mapped[int]` es la anotación de tipo que le dice a SQLAlchemy (y a tu editor) qué tipo de Python corresponde a esa columna; `mapped_column(...)` es donde configurás los detalles de la columna real en la base de datos (tipo SQL, si es clave primaria, si acepta nulos, etc.). El proyecto usa la versión

async de SQLAlchemy 2.0, por lo que todas las consultas se hacen con `await session.execute(...)` en vez de llamadas síncronas.

3.4. Alembic: migraciones versionadas

Cuando cambiás un modelo de SQLAlchemy (agregás un campo, una tabla), la base de datos real no cambia sola — necesitás ejecutar sentencias `ALTER TABLE`, `CREATE TABLE`, etc. Alembic genera y versiona esos cambios como archivos Python (*migraciones*), cada uno con un `upgrade()` (aplicar el cambio) y un `downgrade()` (revertirlo), encadenados entre sí como una lista enlazada (cada migración sabe cuál es la anterior). Volvemos a esto en profundidad en la sección 13.

3.5. ¿Por qué PostgreSQL en producción pero SQLite en los tests?

El proyecto corre contra PostgreSQL real (vía el driver `asyncpg`) para desarrollo y producción, pero los tests automáticos usan SQLite en memoria (vía `aiosqlite`). La razón: como todo el acceso a datos pasa por SQLAlchemy (nunca SQL específico de Postgres fuera de las migraciones), el mismo código de repositorios/servicios funciona contra cualquier motor compatible. Eso permite que los tests corran en milisegundos, sin necesitar Docker ni una base real levantada — el precio es que hay que evitar features específicas de Postgres en el código de la aplicación (sí se pueden usar en las migraciones, que sólo corren contra la base real).

4. Arquitectura general: diseño por capas

El backend sigue una arquitectura **en capas** (también llamada hexagonal o "clean architecture" en sus versiones más elaboradas). La idea central: cada capa sólo puede hablar con la capa inmediatamente debajo suyo, nunca saltearse niveles ni hablar "hacia arriba". Así se ve la estructura de carpetas real del proyecto:

```
app/domain/ # dataclasses puras (Product, Sale, Stock, ...) - sin ORM, sin Pydantic
app/infrastructure/db/
 base.py # DeclarativeBase de SQLAlchemy
 session.py # motor async, sesiones, get_db
 models/ # clases mapeadas de SQLAlchemy, una por entidad, reflejan el dominio 1:1
app/repositories/ # un archivo por entidad: interfaz ABC + implementación SqlAlchemy*
app/schemas/ # DTOs de Pydantic Create/Update/Read - la única capa que conoce la API
app/services/ # lógica de negocio, orquesta repositorios
app/routes/ # routers de FastAPI, uno por entidad, agregados en api_router.py
app/core/ # Settings, excepciones de dominio, mapeo excepción -&gt; HTTP
```

El flujo de una petición típica va así: **ruta** recibe el JSON, lo valida vía **schema** (Pydantic), lo convierte en un objeto de **dominio**, se lo pasa al **service** correspondiente. El service aplica las reglas de negocio y le pide datos al **repository**. El repository traduce entre el objeto de dominio y el **modelo** de SQLAlchemy, y ejecuta la consulta real contra la base. La respuesta hace el camino inverso: modelo -> dominio -> (de vuelta en la ruta) schema de salida -> JSON.

> **La regla de oro:** el dominio (`app/domain/`) nunca importa SQLAlchemy ni Pydantic. Es Python puro. Esto significa que las reglas de negocio (por ejemplo, "una promoción debe apuntar a exactamente un producto o una categoría, no ambos ni ninguno") no dependen de cómo se persisten los datos ni de cómo llegan por HTTP. Si mañana cambiaras de framework web o de ORM, el dominio no se toca.

¿Por qué separar así, en vez de tener una sola clase "Category" que sea a la vez modelo de base de datos, validador de API y contenedor de lógica (como hacen algunos frameworks "todo en uno")? Porque cada responsabilidad cambia por razones distintas: la forma en que validás un JSON de entrada puede cambiar sin que cambie cómo se guarda en la base; el modelo de base de datos puede tener columnas técnicas (índices, claves foráneas) que no tienen nada que ver con la lógica de negocio. Separarlas cuesta más código al principio (cinco archivos por entidad en vez de uno), pero cada archivo es simple y se testea/cambia de forma aislada.

5. El patrón CRUD simple, explicado una sola vez con Category

Seis entidades del proyecto (`Category`, `Brand`, `Color`, `Size`, `Branch`, `PaymentMethod`) son catálogos simples: sólo necesitan crear, leer, actualizar y borrar (CRUD), sin reglas de negocio propias. Vamos a recorrer las cinco capas completas usando `Category`. Cuando en el resto del proyecto veas `Brand`, `Color`, etc., son exactamente este mismo patrón con otros nombres de campo.

5.1. Dominio

`app/domain/category.py`

```
from dataclasses import dataclass


@dataclass(slots=True)
class Category:
   name: str
   description: str | None = None
   id: int | None = None
```

Tres campos: `name` (obligatorio), `description` (opcional) e `id` (opcional, porque un objeto recién construido en memoria todavía no tiene id asignado por la base de datos — recién lo tiene después de guardarlo). Por eso `id` siempre va **último** con default `None`: en un `dataclass`, todos los campos con valor por defecto tienen que ir después de los que no lo tienen, así que la convención del proyecto es "campos obligatorios primero, opcionales después, `id` al final de todos".

5.2. Modelo de SQLAlchemy

`app/infrastructure/db/models/category_model.py`

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from app.infrastructure.db.base import Base


class CategoryModel(Base):
   __tablename__ = "categories"

   id: Mapped[int] = mapped_column(primary_key=True)
   name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
   description: Mapped[str | None] = mapped_column(String(500))
```

Esta clase es *parecida* a `Category` pero no es la misma cosa: acá sí importa detalles de base de datos (`String(100)` es un `VARCHAR(100)` en SQL, `unique=True` agrega una restricción de unicidad, `primary_key=True` marca la clave primaria). `Base` es la clase declarativa compartida por todos los modelos (la vemos en la sección 6.1). El nombre de la tabla real en Postgres es `categories`.

5.3. Repositorio

app/repositories/category_repository.py

```
from abc import ABC, abstractmethod

from app.domain.category import Category
from app.infrastructure.db.models.category_model import CategoryModel
from app.repositories.base_repository import SqlAlchemyRepository


class CategoryRepository(ABC):
   @abstractmethod
   async def list_all(self) -> list[Category]: ...

   @abstractmethod
   async def get_by_id(self, category_id: int) -> Category | None: ...

   @abstractmethod
   async def create(self, category: Category) -> Category: ...

   @abstractmethod
   async def update(self, category: Category) -> Category: ...

   @abstractmethod
   async def delete(self, category_id: int) -> None: ...


class SqlAlchemyCategoryRepository(SqlAlchemyRepository[CategoryModel], CategoryRepository):
   model = CategoryModel

   async def list_all(self) -> list[Category]:
       return [self._to_domain(model) for model in await self._list_models()]

   async def get_by_id(self, category_id: int) -> Category | None:
       model = await self._get_model(category_id)
       return self._to_domain(model) if model else None

   async def create(self, category: Category) -> Category:
       model = CategoryModel(name=category.name, description=category.description)
       self._session.add(model)
       await self._session.flush()
       return self._to_domain(model)

   async def update(self, category: Category) -> Category:
       model = await self._get_model(category.id)
       model.name = category.name
       model.description = category.description
       await self._session.flush()
       return self._to_domain(model)

   async def delete(self, category_id: int) -> None:
       model = await self._get_model(category_id)
       if model is not None:
           await self._delete_model(model)

   @staticmethod
   def _to_domain(model: CategoryModel) -> Category:
       return Category(id=model.id, name=model.name, description=model.description)
```

Puntos clave de este archivo, que se repiten en **todos** los repositorios del proyecto:

- **Dos clases por archivo:** la interfaz abstracta (`CategoryRepository`, la "forma" que cualquier implementación debe tener) y la implementación concreta con SQLAlchemy (`SqlAlchemyCategoryRepository`). Los servicios (sección 5.5) dependen de la interfaz, no de la implementación.
- **`SqlAlchemyRepository[CategoryModel]`:** hereda de la clase base genérica (sección 6.3) pasándole qué modelo de SQLAlchemy le corresponde. Esa base ya trae `_get_model`/`_list_models`/`_delete_model` hechos, así que acá sólo hace falta escribir lo específico de `Category`.
- **`_to_domain`:** convierte un `CategoryModel` (fila de base de datos) en un `Category` (objeto de dominio). Es un `@staticmethod` porque no necesita `self`, sólo transforma datos. Este método existe en *todos* los repositorios y es el único lugar donde se mapean campo por campo un modelo y su dominio equivalente — a mano, sin magia ni reflexión, así que si agregás un campo nuevo tenés que acordarte de tocar `create`, `update` y `_to_domain` los tres.
- **`flush()`, nunca `commit()`:** los repositorios guardan cambios en la sesión con `flush()` (que envía las sentencias SQL pero no cierra la transacción), y dejan el `commit()` final para la capa de infraestructura (sección 6.2). Esto es clave para que una operación que toca varias tablas (como crear una venta con sus ítems y pagos) sea atómica: si algo falla a mitad de camino, se puede revertir todo junto.

5.4. Schema (DTOs de Pydantic)

`app/schemas/category_schema.py`

```
from pydantic import BaseModel, ConfigDict


class CategoryBase(BaseModel):
   name: str
   description: str | None = None


class CategoryCreate(CategoryBase):
   pass


class CategoryUpdate(CategoryBase):
   pass


class CategoryRead(CategoryBase):
   model_config = ConfigDict(from_attributes=True)

   id: int
```

¿Por qué **cuatro** clases para representar "lo mismo"? Porque cada una responde una pregunta distinta:

- `CategoryBase`: los campos que `Create` y `Update` comparten (evita repetir `name`/`description` dos veces).
- `CategoryCreate`: la forma del JSON que el cliente manda para **crear** una categoría. No tiene `id` porque todavía no existe.

- `CategoryUpdate`: la forma del JSON para **actualizar**. En este caso es igual a `Create`, pero en otras entidades del proyecto difieren (por ejemplo, hay campos que se pueden setear al crear pero no modificar después, o viceversa).
- `CategoryRead`: la forma en la que la API **devuelve** una categoría. Sí tiene `id`, porque ya fue persistida. `model_config = ConfigDict(from_attributes=True)` le dice a Pydantic que puede construirse leyendo atributos de un objeto (como nuestro `Category` de dominio) en vez de exigir un diccionario — sin esto, FastAPI no podría convertir el objeto de dominio devuelto por el service en la respuesta JSON automáticamente.

5.5. Servicio

`app/services/category_service.py`

```
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession

from app.domain.category import Category
from app.infrastructure.db.session import get_db
from app.repositories.category_repository import SqlAlchemyCategoryRepository
from app.services.base_service import CrudService


class CategoryService(CrudService[Category]):
   entity_name = "Category"


def get_category_service(session: AsyncSession = Depends(get_db)) -> CategoryService:
   return CategoryService(SqlAlchemyCategoryRepository(session))
```

Esta clase es sorprendentemente corta: hereda **todo** su comportamiento (`list_all`/`get_by_id`/`create`/`update`/`delete`) de `CrudService[Category]` (la explicamos línea por línea en la sección 6.4), y sólo define `entity_name = "Category"` (usado para mensajes de error tipo "Category with id=5 was not found").

Al final del archivo aparece el patrón de **wiring de dependencias** que se repite en cada módulo de servicio del proyecto: una función `get_<entidad>_service` que recibe una sesión de base de datos (inyectada por FastAPI vía `Depends(get_db)`) y devuelve una instancia del service, ya armada con su repositorio concreto adentro. No hay un archivo central de "inyección de dependencias" — cada módulo arma las suyas, al lado de la clase que las usa.

5.6. Ruta

`app/routes/category_route.py`

```
from fastapi import APIRouter, Depends, status

from app.domain.category import Category
from app.schemas.category_schema import CategoryCreate, CategoryRead, CategoryUpdate
from app.services.category_service import CategoryService, get_category_service

router = APIRouter(prefix="/categories", tags=["Categories"])
```

```
@router.get("", response_model=list[CategoryRead])
async def list_categories(
   service: CategoryService = Depends(get_category_service),
) -> list[Category]:
   return await service.list_all()


@router.get("/{category_id}", response_model=CategoryRead)
async def get_category(
   category_id: int, service: CategoryService = Depends(get_category_service)
) -> Category:
   return await service.get_by_id(category_id)


@router.post("", response_model=CategoryRead, status_code=status.HTTP_201_CREATED)
async def create_category(
   payload: CategoryCreate, service: CategoryService = Depends(get_category_service)
) -> Category:
   return await service.create(Category(**payload.model_dump()))


@router.put("/{category_id}", response_model=CategoryRead)
async def update_category(
   category_id: int,
   payload: CategoryUpdate,
   service: CategoryService = Depends(get_category_service),
) -> Category:
   return await service.update(category_id, Category(**payload.model_dump()))


@router.delete("/{category_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_category(
   category_id: int, service: CategoryService = Depends(get_category_service)
) -> None:
   await service.delete(category_id)
```

Notas sobre este archivo:

- `APIRouter(prefix="/categories", tags=["Categories"])`: agrupa todos los endpoints bajo `/categories`, y `tags` es sólo para organizar la documentación automática (Swagger, en `/docs`).
- `response_model=...`: le dice a FastAPI qué schema Pydantic usar para **serializar la respuesta**. Aunque la función devuelve un objeto `Category` de dominio, FastAPI lo convierte a `CategoryRead` (gracias al `from_attributes=True` de la sección 5.4) antes de mandarlo como JSON. Esto también filtra: si `Category` tuviera un campo interno que no debería salir en la API, simplemente no lo pondrías en `CategoryRead` y nunca se filtraría.
- `Category(**payload.model_dump())`: convierte el schema Pydantic ya validado en un objeto de dominio. `payload.model_dump()` lo pasa a diccionario, y `**` lo "desempaqueta" como argumentos nombrados al constructor de `Category`.
- Los códigos de estado HTTP se declaran explícitamente donde no son el default de FastAPI (200): `201 Created` al crear, `204 No Content` al borrar (sin cuerpo de respuesta).

Esto es todo el patrón. Brand, Color, Size, Branch y PaymentMethod son copias de estos cinco archivos con otro nombre de entidad y otros campos. Si en algún momento tenés que agregar una entidad de catálogo simple nueva, literalmente podés copiar estos cinco archivos de `Category`, hacer

> buscar-y-reemplazar del nombre, y ajustar los campos.

6. Infraestructura core, a fondo

Vimos cómo se ve **una** entidad completa. Ahora veamos las piezas compartidas que hacen posible ese patrón: la base declarativa de SQLAlchemy, el manejo de sesiones (y por qué el commit vive en un solo lugar), las clases genéricas base de repositorio y servicio, y cómo se traducen los errores de negocio a respuestas HTTP.

6.1. La base declarativa

`app/infrastructure/db/base.py`

```
from sqlalchemy.orm import DeclarativeBase


class Base(DeclarativeBase):
   """Declarative base shared by every ORM model. Import all models before
   running Alembic autogenerate so they register on this metadata."""
```

Todos los modelos (`CategoryModel`, `ProductModel`, etc.) heredan de esta misma clase `Base`. Al heredar, SQLAlchemy los registra automáticamente en `Base.metadata`, un catálogo interno de "todas las tablas que existen". Ese catálogo es justamente lo que Alembic lee para comparar "cómo debería ser el esquema" contra "cómo es la base real" y generar migraciones (sección 13). El comentario del docstring ya adelanta un detalle importante: si un modelo no se *importa* en algún punto del programa antes de correr Alembic, Python nunca ejecuta esa clase y nunca se registra — por eso `alembic/env.py` y `tests/conftest.py` importan expresamente todos los módulos de modelos, aunque no usen sus nombres directamente (fijate el comentario `# noqa: F401`, que le dice al linter "sé que este import no se usa directamente, es intencional").

6.2. Sesiones y el patrón "unit of work"

`app/infrastructure/db/session.py`

```
from collections.abc import AsyncGenerator

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

from app.core.config import get_settings

settings = get_settings()

engine = create_async_engine(settings.database_url, echo=settings.debug, future=True)

async_session_factory = async_sessionmaker(
   bind=engine,
   autoflush=False,
   expire_on_commit=False,
)


async def get_db() -> AsyncGenerator[AsyncSession, None]:
   """Yields a session scoped to a single request; commits on success,
   rolls back on error. Repositories only flush, never commit."""
   async with async_session_factory() as session:
```

```
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

`engine` es la conexión "de fábrica" a la base de datos (sabe cómo hablar con Postgres vía `asyncpg`, pero no mantiene una conexión abierta todo el tiempo, las administra en un pool). `async_session_factory` es una fábrica de *sesiones*: cada sesión representa una transacción de trabajo con la base.

`get_db` es una **función generadora** (usa `yield` en vez de `return`). FastAPI sabe reconocer esto cuando la usás con `Depends(get_db)`: ejecuta el código hasta el `yield` (abre la sesión), le entrega esa sesión a tu función de ruta o servicio, espera a que la petición termine, y **después** retoma la ejecución después del `yield` (hace `commit()` o `rollback()` según haya habido excepción). Esto se llama patrón **unit of work**: todo lo que pasa durante una petición HTTP usa la misma sesión/transacción, y se confirma o se revierte *todo junto*, en un único lugar.

> **Por qué esto importa para operaciones complejas:** crear una venta implica insertar una fila en `sales`, varias en `sale_items`, varias en `sale_payments`, y actualizar cantidades en `stocks`. Si a mitad de camino algo falla (por ejemplo, no hay stock suficiente), **no** queremos que la venta quede guardada a medias. Como todos los repositorios que participan en `SaleService.create_sale` comparten la misma sesión (porque `get_sale_service` construye todos sus repositorios pasándoles la misma `session` inyectada), y esa sesión sólo hace `commit()` al final en `get_db`, un error en cualquier punto revierte *todo* lo hecho hasta ahí. Los repositorios individuales sólo hacen `flush()` (mandan el SQL a la base para que ya puedan usarse los ids generados, sin cerrar la transacción) — nunca `commit()`.

6.3. El repositorio genérico

`app/repositories/base_repository.py`

```
from typing import Generic, TypeVar

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

ModelT = TypeVar("ModelT")


class SqlAlchemyRepository(Generic[ModelT]):
   """Generic persistence helpers shared by the simple, single-table
   repositories. Subclasses set `model` and add the domain&lt;-&gt;model mapping
   plus any query specific to that entity."""

   model: type[ModelT]

   def __init__(self, session: AsyncSession) -> None:
       self._session = session

   async def _get_model(self, entity_id: int) -> ModelT | None:
       return await self._session.get(self.model, entity_id)

   async def _list_models(self) -> list[ModelT]:
       result = await self._session.execute(select(self.model))
       return list(result.scalars().all())
```

```
    async def _delete_model(self, instance: ModelT) -> None:
        await self._session.delete(instance)
        await self._session.flush()
```

Esto es lo que `SqlAlchemyCategoryRepository(SqlAlchemyRepository[CategoryModel], ...)` hereda (sección 5.3). `ModelT` es el genérico (sección 2.5): cuando una subclase escribe `SqlAlchemyRepository[CategoryModel]`, está diciendo "en esta instancia, cada vez que aparezca `ModelT` en el código de la clase base, tratalo como `CategoryModel`". Así `_get_model` devuelve `CategoryModel | None` con seguridad de tipos, sin escribir esa lógica de nuevo en cada repositorio.

- `session.get(self.model, entity_id)`: la forma más directa de SQLAlchemy de buscar una fila por clave primaria (equivale a `SELECT * FROM tabla WHERE id = ?`, pero optimizado internamente).
- `select(self.model)` + `session.execute(...)`: la forma general de armar consultas en SQLAlchemy 2.0 (el "lenguaje de expresiones"). `result.scalars().all()` extrae la lista de objetos del resultado.
- Métodos con guion bajo adelante (`_get_model`) son una convención de Python para "esto es un detalle interno, no forma parte de la interfaz pública" — no es una restricción real del lenguaje (Python no tiene `private` de verdad), es una señal para quien lea el código.

6.4. El servicio genérico

app/services/base_service.py

```
from typing import Generic, Protocol, TypeVar

from app.core.exceptions import EntityNotFoundError

DomainT = TypeVar("DomainT")


class CrudRepositoryProtocol(Protocol[DomainT]):
    async def list_all(self) -> list[DomainT]: ...
    async def get_by_id(self, entity_id: int) -> DomainT | None: ...
    async def create(self, entity: DomainT) -> DomainT: ...
    async def update(self, entity: DomainT) -> DomainT: ...
    async def delete(self, entity_id: int) -> None: ...


class CrudService(Generic[DomainT]):
    """Business logic shared by the simple catalog entities: existence
    checks around an otherwise pass-through repository. Entities with real
    business rules (e.g. Sale) do not use this base class."""

    entity_name: str

    def __init__(self, repository: CrudRepositoryProtocol[DomainT]) -> None:
        self._repository = repository

    async def list_all(self) -> list[DomainT]:
        return await self._repository.list_all()

    async def get_by_id(self, entity_id: int) -> DomainT:
        entity = await self._repository.get_by_id(entity_id)
```

```
        if entity is None:
            raise EntityNotFoundError(self.entity_name, entity_id)
        return entity

    async def create(self, entity: DomainT) -> DomainT:
        return await self._repository.create(entity)

    async def update(self, entity_id: int, entity: DomainT) -> DomainT:
        await self.get_by_id(entity_id)
        entity.id = entity_id # type: ignore[attr-defined]
        return await self._repository.update(entity)

    async def delete(self, entity_id: int) -> None:
        await self.get_by_id(entity_id)
        await self._repository.delete(entity_id)
```

Acá aparece `Protocol` (sección 2.6) en `CrudRepositoryProtocol`: en vez de exigir que el repositorio herede de una clase concreta, `CrudService` sólo pide que tenga esos cinco métodos con esas firmas. Cualquier repositorio de catálogo simple (Category, Brand, etc.) cumple ese contrato "por estructura", sin que `CrudService` tenga que conocer sus clases concretas.

El único método con lógica no trivial es `get_by_id`: si el repositorio devuelve `None` (no encontrado), **acá** es donde se convierte esa ausencia en una excepción de dominio (`EntityNotFoundError`, sección 6.5) con el nombre de la entidad y el id que se buscó. Fijate que `update` y `delete` **reusan** `get_by_id` primero: así, si el id no existe, el error sale antes de intentar nada más, con un mensaje consistente en todas las entidades.

> **El docstring lo dice explícitamente:** "Entities with real business rules (e.g. Sale) do not use this base class". Esta clase resuelve el 80% de los casos (las entidades de catálogo), pero no pretende cubrir todo. Cuando una entidad tiene reglas propias (validar claves foráneas, calcular totales, etc.), o bien *extiende* este comportamiento sobrescribiendo `create`/ `update` (como vas a ver con Product y Stock), o bien prescinde de esta clase base por completo y escribe su propio service desde cero (como Sale, Purchase y Promotion). Reconocer cuándo conviene cada camino es una de las decisiones de diseño más importantes al agregar una entidad nueva — volvemos a esto en la sección 14.

6.5. Excepciones de dominio y su traducción a HTTP

`app/core/exceptions.py`

```
class DomainError(Exception):
    """Base class for errors raised by the domain/service layer."""


class EntityNotFoundError(DomainError):
    """Raised when a lookup by id does not match any record."""

    def __init__(self, entity_name: str, entity_id: object) -> None:
        self.entity_name = entity_name
        self.entity_id = entity_id
        super().__init__(f"{entity_name} with id={entity_id!r} was not found")


class InsufficientStockError(DomainError):
    """Raised when a sale requests more units than are available in stock."""
```

```
    def __init__(self, stock_id: int, requested: int, available: int) -> None:
        self.stock_id = stock_id
        self.requested = requested
        self.available = available
        super().__init__(
            f"Stock {stock_id} has {available} unit(s) available, {requested} requested"
        )


class BusinessRuleViolationError(DomainError):
    """Raised when an operation would break a domain invariant (e.g. totals mismatch)."""
```

Tres tipos de error, todos heredando de `DomainError` (que a su vez hereda de la `Exception` estándar de Python). Ninguno sabe nada de HTTP — son excepciones de Python comunes, con datos extra guardados como atributos (`self.entity_name`, `self.stock_id`, etc.) para que quien las capture pueda armar un mensaje útil sin tener que parsear texto.

`app/core/exception_handlers.py`

```
from fastapi import FastAPI, Request, status
from fastapi.responses import JSONResponse

from app.core.exceptions import (
    BusinessRuleViolationError,
    EntityNotFoundError,
    InsufficientStockError,
)


async def entity_not_found_handler(_: Request, exc: EntityNotFoundError) -> JSONResponse:
    return JSONResponse(status_code=status.HTTP_404_NOT_FOUND, content={"detail": str(exc)})


async def insufficient_stock_handler(_: Request, exc: InsufficientStockError) ->
JSONResponse:
    return JSONResponse(status_code=status.HTTP_409_CONFLICT, content={"detail": str(exc)})


async def business_rule_violation_handler(
    _: Request, exc: BusinessRuleViolationError
) -> JSONResponse:
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY, content={"detail": str(exc)}
    )


def register_exception_handlers(app: FastAPI) -> None:
    app.add_exception_handler(EntityNotFoundError, entity_not_found_handler)
    app.add_exception_handler(InsufficientStockError, insufficient_stock_handler)
    app.add_exception_handler(BusinessRuleViolationError, business_rule_violation_handler)
```

Acá está el único lugar de todo el proyecto que traduce excepciones de dominio a códigos HTTP: `EntityNotFoundError` → 404, `InsufficientStockError` → 409 (Conflict, porque el pedido en sí es válido pero choca con el estado actual del sistema), `BusinessRuleViolationError` → 422 (Unprocessable Entity, "entendí el JSON pero viola una regla de negocio"). FastAPI permite registrar handlers globales por tipo de excepción con `app.add_exception_handler`: si en cualquier service o repositorio se lanza una de estas excepciones (con un simple `raise EntityNotFoundError(...)`, sin

importar `HTTPException` ni nada de FastAPI), termina devolviendo la respuesta HTTP correcta automáticamente.

> **Por qué esto es mejor que usar `HTTPException` directamente en cada service:** si los services lanzaran `HTTPException(status_code=404, ...)` a mano, estarían acoplados a FastAPI (y no se podrían testear ni reusar fuera de un contexto HTTP). Con excepciones de dominio + handlers centralizados, un service se puede probar con un test unitario común (sección 12) que simplemente verifica "se lanzó `EntityNotFoundError`", sin necesidad de un servidor HTTP corriendo.

6.6. El punto de ensamblaje: `main.py` y `api_router.py`

app/main.py

```
from fastapi import FastAPI

from app.core.config import get_settings
from app.core.exception_handlers import register_exception_handlers
from app.routes.api_router import api_router


def create_app() -> FastAPI:
   settings = get_settings()
   app = FastAPI(title=settings.app_name, debug=settings.debug)

   register_exception_handlers(app)
   app.include_router(api_router)

   return app


app = create_app()
```

`create_app()` es una *factory function*: en vez de crear la instancia de `FastAPI` directamente a nivel de módulo, la arma dentro de una función. Esto facilita testear (en `tests/conftest.py` se puede importar `app` y sobreescribir sus dependencias sin afectar una instancia global compartida por accidente) y deja un único lugar donde se "conecta todo": se registran los manejadores de excepciones y se incluye el router principal.

app/routes/api_router.py

```
from fastapi import APIRouter

from app.routes.branch_route import router as branch_router
from app.routes.brand_route import router as brand_router
from app.routes.category_route import router as category_router
# ... (uno por cada entidad)

api_router = APIRouter()

api_router.include_router(category_router)
api_router.include_router(brand_router)
# ... (en el mismo orden, aproximadamente de menor a mayor dependencia)
api_router.include_router(sale_router)
api_router.include_router(exchange_router)
```

Cada archivo de rutas (`category_route.py`, `product_route.py`, etc.) define su propio `router` con su `prefix`. `api_router.py` los junta todos en un único router raíz, que es lo único que `main.py` necesita conocer. Agregar una entidad nueva a la API es, literalmente, agregar dos líneas acá.

7. Caso de estudio: Product

`Product` tiene la misma forma de cinco archivos que `Category`, pero es la primera entidad donde el `service` hace algo más que delegar en `CrudService`: valida que las claves foráneas (`category_id`, `brand_id`) realmente existan antes de guardar.

app/domain/product.py

```
from dataclasses import dataclass
from decimal import Decimal


@dataclass(slots=True)
class Product:
   name: str
   category_id: int
   brand_id: int
   cost: Decimal
   price: Decimal
   description: str | None = None
   image_url: str | None = None
   id: int | None = None
```

¿Por qué `Decimal` y no `float` para el dinero? `float` representa números en binario, y valores como `0.1` no tienen representación exacta en binario — sumar muchos `float` "redondos" en decimal puede dar resultados con error de centavos (`0.1 + 0.2 == 0.30000000000000004` en Python, probalo). `Decimal` (del módulo estándar `decimal`) representa los números exactamente en base 10, como esperarías al hacer cuentas con plata a mano. Todo el dinero del proyecto (precios, costos, montos de pago, descuentos) usa `Decimal`, nunca `float`.

app/services/product_service.py

```
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.exceptions import EntityNotFoundError
from app.domain.product import Product
from app.infrastructure.db.session import get_db
from app.repositories.brand_repository import BrandRepository, SqlAlchemyBrandRepository
from app.repositories.category_repository import CategoryRepository,
SqlAlchemyCategoryRepository
from app.repositories.product_repository import ProductRepository,
SqlAlchemyProductRepository
from app.services.base_service import CrudService


class ProductService(CrudService[Product]):
   entity_name = "Product"

   def __init__(
       self,
       repository: ProductRepository,
       category_repository: CategoryRepository,
       brand_repository: BrandRepository,
   ) -> None:
       super().__init__(repository)
```

```
        self._category_repository = category_repository
        self._brand_repository = brand_repository

    async def create(self, product: Product) -> Product:
        await self._ensure_references_exist(product)
        return await super().create(product)

    async def update(self, entity_id: int, product: Product) -> Product:
        await self._ensure_references_exist(product)
        return await super().update(entity_id, product)

    async def _ensure_references_exist(self, product: Product) -> None:
        if await self._category_repository.get_by_id(product.category_id) is None:
            raise EntityNotFoundError("Category", product.category_id)
        if await self._brand_repository.get_by_id(product.brand_id) is None:
            raise EntityNotFoundError("Brand", product.brand_id)


def get_product_service(session: AsyncSession = Depends(get_db)) -> ProductService:
    return ProductService(
        SqlAlchemyProductRepository(session),
        SqlAlchemyCategoryRepository(session),
        SqlAlchemyBrandRepository(session),
    )
```

Este archivo enseña un patrón que se repite en todo el proyecto cada vez que una entidad depende de otras:

- **El constructor recibe repositorios "hermanos".** `ProductService` no sólo recibe su propio `ProductRepository`, sino también `CategoryRepository` y `BrandRepository`, sólo para poder *consultarlos* (nunca los usa para escribir, sólo `get_by_id`).
- **`super().__init__(repository)`** llama al constructor de la clase padre (`CrudService`) para que guarde `self._repository`, y *después* guarda los repositorios extra en atributos propios.
- **Sobrescribir `create`/`update` sin duplicar lógica:** ambos métodos llaman primero a `_ensure_references_exist` (la validación nueva), y *después* delegan en `super().create(product)`/`super().update(entity_id, product)` — el comportamiento original de `CrudService` (guardar en el repositorio) sigue intacto, sólo se le agregó un paso previo. Esto es el principio de **abrir para extensión, cerrar para modificación**: no tocamos el código de `CrudService`, lo extendemos desde afuera.

El resto del archivo (`app/repositories/product_repository.py`, el modelo, el schema, la ruta) sigue exactamente el mismo patrón que `Category`, con los campos propios de Product mapeados. No hace falta repetirlo acá.

8. Caso de estudio: Stock

`Stock` representa "cuántas unidades de tal producto, en tal color y talle, hay en tal sucursal". Es otra entidad con validación cruzada de FKs (como Product), pero agrega dos cosas nuevas: campos opcionales por una razón de negocio concreta, y operaciones que no son CRUD puro (sumar/restar cantidad).

`app/domain/stock.py`

```
from dataclasses import dataclass


@dataclass(slots=True)
class Stock:
   product_id: int
   branch_id: int
   quantity: int
   color_id: int | None = None
   size_id: int | None = None
   id: int | None = None
```

`color_id` y `size_id` son opcionales: no todos los productos tienen variedad de color o talle (por ejemplo, un producto único). Fijate el **orden** de los campos: `product_id`, `branch_id` y `quantity` van primero porque son obligatorios (sin default); recién después los opcionales, porque un `dataclass` exige que todo campo con valor por defecto venga después de los que no lo tienen — si intentaras poner `color_id: int | None = None` antes que `branch_id: int` (sin default), Python tira un error en tiempo de definición de la clase, no en tiempo de uso.

`app/infrastructure/db/models/stock_model.py`

```
from sqlalchemy import ForeignKey, Integer, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column

from app.infrastructure.db.base import Base


class StockModel(Base):
   __tablename__ = "stocks"
   __table_args__ = (
       UniqueConstraint("product_id", "color_id", "size_id", "branch_id",
name="uq_stock_variant"),
   )

   id: Mapped[int] = mapped_column(primary_key=True)
   product_id: Mapped[int] = mapped_column(ForeignKey("products.id"), nullable=False)
   color_id: Mapped[int | None] = mapped_column(ForeignKey("colors.id"), nullable=True)
   size_id: Mapped[int | None] = mapped_column(ForeignKey("sizes.id"), nullable=True)
   branch_id: Mapped[int] = mapped_column(ForeignKey("branches.id"), nullable=False)
   quantity: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
```

`__table_args__` es donde SQLAlchemy permite declarar restricciones que involucran más de una columna. Acá, `UniqueConstraint(...)` le dice a Postgres "no puede haber dos filas con la misma combinación de producto+color+talle+sucursal" — evita crear stock duplicado por error para la misma variante.

> **Un detalle sutil de SQL que vale la pena conocer:** en Postgres (y en SQL en general), un `UNIQUE CONSTRAINT` considera que `NULL` nunca es igual a otro `NULL`. Como `color_id`/`size_id` ahora pueden ser `NULL`, *sí se pueden crear varias filas* de stock para el mismo producto+sucursal con color y talle en `NULL` — la restricción de unicidad no las detecta como duplicadas. Es una limitación conocida y aceptada del diseño actual, no un bug: si en algún momento importara evitar eso, se necesitaría un índice único parcial de Postgres (algo más avanzado, fuera del alcance de esta guía).

`app/services/stock_service.py`

```
class StockService(CrudService[Stock]):
    entity_name = "Stock"

    def __init__(
        self,
        repository: StockRepository,
        product_repository: ProductRepository,
        color_repository: ColorRepository,
        size_repository: SizeRepository,
        branch_repository: BranchRepository,
    ) -> None:
        super().__init__(repository)
        self._product_repository = product_repository
        self._color_repository = color_repository
        self._size_repository = size_repository
        self._branch_repository = branch_repository

    async def create(self, stock: Stock) -> Stock:
        await self._ensure_references_exist(stock)
        return await super().create(stock)

    async def update(self, entity_id: int, stock: Stock) -> Stock:
        await self._ensure_references_exist(stock)
        return await super().update(entity_id, stock)

    async def decrease_quantity(self, stock_id: int, amount: int) -> Stock:
        stock = await self.get_by_id(stock_id)
        if stock.quantity < amount:
            raise InsufficientStockError(stock_id, amount, stock.quantity)
        stock.quantity -= amount
        return await self._repository.update(stock)

    async def increase_quantity(self, stock_id: int, amount: int) -> Stock:
        stock = await self.get_by_id(stock_id)
        stock.quantity += amount
        return await self._repository.update(stock)

    async def _ensure_references_exist(self, stock: Stock) -> None:
        if await self._product_repository.get_by_id(stock.product_id) is None:
            raise EntityNotFoundError("Product", stock.product_id)
        if stock.color_id is not None:
            if await self._color_repository.get_by_id(stock.color_id) is None:
                raise EntityNotFoundError("Color", stock.color_id)
        if stock.size_id is not None:
            if await self._size_repository.get_by_id(stock.size_id) is None:
                raise EntityNotFoundError("Size", stock.size_id)
        if await self._branch_repository.get_by_id(stock.branch_id) is None:
            raise EntityNotFoundError("Branch", stock.branch_id)
```

Tres cosas para notar acá:

- `_ensure_references_exist` valida `color_id`/`size_id` sólo `if stock.color_id is not None`. Si no lo hiciera, un stock sin color explotaría con un error raro al intentar buscar "el color con id `None`". Este es exactamente el tipo de detalle que hay que recordar ajustar cuando un campo pasa de obligatorio a opcional en cualquier parte del sistema.
- `increase_quantity`/`decrease_quantity` son métodos *de negocio*, no parte del CRUD genérico: llaman a `self.get_by_id(stock_id)` (heredado de `CrudService`, que ya lanza `EntityNotFoundError` si no existe), modifican el objeto de dominio en memoria, y lo guardan con `self._repository.update(stock)` directamente (no pasan por `super().update()` porque no hace falta repetir la validación de referencias: el producto/color/talle/sucursal ya eran válidos cuando se creó el stock, sólo cambia la cantidad).
- `decrease_quantity` valida `stock.quantity < amount` **antes** de restar, y lanza `InsufficientStockError` (409) si no alcanza — esta es la validación que impide vender más de lo que hay.

La ruta (`app/routes/stock_route.py`) agrega dos endpoints además de los cinco CRUD estándar, justamente para exponer estas dos operaciones:

```
@router.post("/{stock_id}/increase", response_model=StockRead)
async def increase_stock(
   stock_id: int,
   payload: StockQuantityAdjustment,
   service: StockService = Depends(get_stock_service),
) -> Stock:
   return await service.increase_quantity(stock_id, payload.amount)


@router.post("/{stock_id}/decrease", response_model=StockRead)
async def decrease_stock(
   stock_id: int,
   payload: StockQuantityAdjustment,
   service: StockService = Depends(get_stock_service),
) -> Stock:
   return await service.decrease_quantity(stock_id, payload.amount)
```

`StockQuantityAdjustment` es un schema aparte (sólo `amount: int`, validado con `Field(gt=0)` para no aceptar cantidades cero o negativas) — no reusa `StockCreate`/`StockUpdate` porque estas dos rutas no reciben "un stock entero", sólo "cuánto sumar o restar".

9. Caso de estudio a fondo: Sale

`Sale` (una venta) es el flujo más complejo del sistema: no es una fila suelta, es un **agregado** — una venta "padre" con varias líneas (`SaleItem`, uno por producto vendido) y varios pagos (`SalePayment`, porque una venta puede pagarse con más de un medio de pago). Además, crear una venta dispara reglas de negocio encadenadas: calcular totales, decidir qué promociones aplican, verificar que lo pagado coincida exactamente, y descontar del stock. Todo esto tiene que pasar **o todo, o nada** (recordá la sección 6.2 sobre unit of work).

9.1. Dominio: un agregado, no una sola clase

app/domain/sale.py

```
from dataclasses import dataclass
from decimal import Decimal


@dataclass(slots=True)
class Sale:
   branch_id: int
   subtotal: Decimal
   discount: Decimal
   total: Decimal
   promotion_discount: Decimal = Decimal("0")
   discount_reason: str | None = None
   id: int | None = None
```

app/domain/sale_item.py

```
from dataclasses import dataclass
from decimal import Decimal


@dataclass(slots=True)
class SaleItem:
   stock_id: int
   quantity: int
   unit_price: Decimal
   promotion_id: int | None = None
   sale_id: int | None = None
   id: int | None = None
```

Notá que `SaleItem` no tiene `subtotal` ni ningún campo calculado: se recalcula siempre a partir de `unit_price * quantity` cuando hace falta, nunca se guarda un total redundante (que se podría desincronizar del dato real si alguien lo actualiza mal). Y `promotion_id` es `None` por default: significa "todavía no se sabe, o no aplica ninguna promoción a esta línea" — lo completa el service, nunca el cliente de la API (volvemos a esto en 9.3).

`SalePayment` (no reproducido acá, mismo patrón) tiene `payment_method_id`, `amount_without_surcharge` y `amount_with_surcharge`: se guardan los dos montos porque algunos medios de pago (tarjeta de crédito, por ejemplo) pueden tener un recargo, y el sistema necesita saber cuánto es "el precio real" versus "cuánto termina pagando el cliente con recargo incluido".

9.2. El schema compuesto: `SaleCreate`

app/schemas/sale_schema.py

```
class SaleItemInput(BaseModel):
   stock_id: int
   quantity: int = Field(gt=0)
   unit_price: Decimal = Field(ge=0)


class SalePaymentInput(BaseModel):
   payment_method_id: int
   amount_without_surcharge: Decimal = Field(ge=0)
   amount_with_surcharge: Decimal = Field(ge=0)


class SaleCreate(BaseModel):
   branch_id: int
   discount: Decimal = Field(default=Decimal("0"), ge=0)
   discount_reason: str | None = None
   items: list[SaleItemInput] = Field(min_length=1)
   payments: list[SalePaymentInput] = Field(min_length=1)
```

A diferencia de `CategoryCreate` (que es plano), `SaleCreate` es **compuesto**: contiene listas anidadas de otros schemas (`items`, `payments`). Pydantic valida la estructura completa de una sola vez — si el JSON que llega tiene un ítem sin `stock_id`, o una cantidad negativa, la petición entera se rechaza con un 422 antes de que el service vea nada. `Field(min_length=1)` exige que haya al menos un ítem y al menos un pago; no tendría sentido una venta vacía.

Fijate qué campos NO están acá: `subtotal`, `total` y `promotion_discount` no existen en `SaleCreate`. El cliente de la API nunca los manda — se **calculan en el servidor**. Si el cliente pudiera mandar el total directamente, alguien podría falsificar una venta con un total menor al real. Este es un principio importante: cualquier dato que sea resultado de una regla de negocio (no una entrada del usuario) debería calcularse del lado del servidor, nunca confiar en lo que manda el cliente. Lo mismo pasa con `SaleItem.promotion_id`: no está en `SaleItemInput` (el cliente no elige qué promoción se aplica), sólo en `SaleItemRead` (la respuesta, después de que el servidor decidió).

9.3. El corazón del sistema: `SaleService.create_sale`

app/services/sale_service.py

```
async def create_sale(
   self,
   branch_id: int,
   discount: Decimal,
   items: list[SaleItem],
   payments: list[SalePayment],
   discount_reason: str | None = None,
) -> Sale:
   subtotal = sum((item.unit_price * item.quantity for item in items), Decimal("0"))

   cart_quantities: dict[int, int] = {}
   cart_categories: dict[int, int] = {}
   cart_unit_prices: dict[int, Decimal] = {}
   cart_item_index: dict[int, list[SaleItem]] = {}
```

```
    for item in items:
        stock = await self._stock_service.get_by_id(item.stock_id)
        product = await self._product_repository.get_by_id(stock.product_id)
        if product is None:
            raise EntityNotFoundError("Product", stock.product_id)
        cart_quantities[product.id] = cart_quantities.get(product.id, 0) + item.quantity
        cart_categories.setdefault(product.id, product.category_id)
        cart_unit_prices.setdefault(product.id, item.unit_price)
        cart_item_index.setdefault(product.id, []).append(item)

    today = date.today()
    applications = await self._promotion_service.select_winning_applications(
        cart_quantities, cart_categories, cart_unit_prices, today
    )

    promotion_discount = Decimal("0")
    for application in applications:
        remaining_to_consume = application.quantity
        for cart_item in cart_item_index.get(application.product_id, []):
            if remaining_to_consume <= 0:
                break
            consumed = min(cart_item.quantity, remaining_to_consume)
            if consumed > 0:
                cart_item.promotion_id = application.promotion_id
                remaining_to_consume -= consumed
        promotion_discount += application.discount_amount

    total = subtotal - discount - promotion_discount
    if total < 0:
        raise BusinessRuleViolationError("Discount cannot exceed the sale subtotal")

    paid_total = sum((payment.amount_with_surcharge for payment in payments), Decimal("0"))
    if paid_total != total:
        raise BusinessRuleViolationError(
            f"Payments total ({paid_total}) does not match the sale total ({total})"
        )

    for payment in payments:
        if await self._payment_method_repository.get_by_id(payment.payment_method_id) is
None:
            raise EntityNotFoundError("PaymentMethod", payment.payment_method_id)

    sale = await self._sale_repository.create(
        Sale(
            branch_id=branch_id,
            subtotal=subtotal,
            discount=discount,
            promotion_discount=promotion_discount,
            discount_reason=discount_reason,
            total=total,
        )
    )

    for item in items:
        await self._stock_service.decrease_quantity(item.stock_id, item.quantity)
        item.sale_id = sale.id
        created_item = await self._sale_item_repository.create(item)
        item.id = created_item.id

    for payment in payments:
        payment.sale_id = sale.id
```

```
        created_payment = await self._sale_payment_repository.create(payment)
        payment.id = created_payment.id

    return sale
```

Recorramos el método paso a paso, porque cada bloque resuelve un problema concreto:

Paso 1 — Subtotal

`sum((item.unit_price * item.quantity for item in items), Decimal("0"))` suma el subtotal de cada línea. El segundo argumento de `sum()` (`Decimal("0")`) es el valor inicial: sin esto, `sum()` empezaría desde el `int` `0`, y sumar `Decimal` a `int` puede dar problemas de tipo en algunos contextos — arrancar explícitamente desde un `Decimal("0")` evita ambigüedad.

Paso 2 — Armar el "resumen del carrito"

Esta es la parte nueva de la Fase 2 (el rediseño de promociones). Por cada línea del carrito, el código resuelve `Stock → Product` (necesita saber a qué producto y categoría pertenece cada línea), y va construyendo **cuatro diccionarios** indexados por `product_id`:

- `cart_quantities`: cuántas unidades de cada producto hay en total en el carrito (sumando entre líneas, si el mismo producto aparece en más de una línea — por ejemplo, dos variantes de talle del mismo producto).
- `cart_categories`: a qué categoría pertenece cada producto (hace falta para las promociones que apuntan a una categoría entera, no a un producto puntual).
- `cart_unit_prices`: el precio unitario de cada producto (se toma el precio de la primera línea de ese producto que aparece en el carrito).
- `cart_item_index`: la lista de `SaleItem` reales de cada producto, en orden — hace falta después, en el Paso 4, para saber a qué línea concreta marcarle el `promotion_id`.

Por qué "resumir por producto" y no trabajar directamente línea por línea: porque las condiciones de una promoción (sección 11) están definidas en términos de "cuántas unidades de tal producto", no de "tal línea puntual" — si el mismo producto está repartido en dos líneas del carrito (dos talles distintos, por ejemplo), la promoción debería considerar la cantidad total, no cada línea por separado.

Paso 3 — Preguntarle a Promotion qué gana

`await self._promotion_service.select_winning_applications(...)` es una única llamada que le delega *todo* el problema de "qué promociones aplican y cuál gana" al service de Promotion (sección 11.4). `SaleService` no necesita saber nada sobre cómo se resuelve eso — sólo le pasa el resumen del carrito y recibe una lista de `PromotionApplication` (cada una dice "esta promoción, descuenta tanto, en este producto, por esta cantidad"). Esto es **separación de responsabilidades**: la lógica de "cuál promoción gana" vive enteramente en `PromotionService`, no mezclada acá.

Paso 4 — Repartir el descuento en las líneas reales

Por cada `PromotionApplication` ganadora, el código recorre `cart_item_index[product_id]` (las líneas reales de ese producto, en el orden en que aparecieron) y les va marcando `cart_item.promotion_id = application.promotion_id` hasta cubrir la `quantity` que indica la aplicación. `min(cart_item.quantity, remaining_to_consume)` calcula cuánto de *esta* línea se

"consume" contra lo que falta cubrir; el `break` corta el loop apenas ya se cubrió toda la cantidad. Al final, `promotion_discount` acumula la suma de todos los descuentos de todas las promociones ganadoras.

Paso 5 — Validar el total y los pagos

`total = subtotal - discount - promotion_discount`: el descuento manual (`discount`, con su `discount_reason` opcional) y el de promociones se restan juntos. Si el resultado fuera negativo, es una venta inválida (`BusinessRuleViolationError`). Después, `paid_total` suma lo que efectivamente se está pagando (con recargos incluidos, si los hay) y **debe coincidir exactamente** con `total` — ni de más ni de menos. Esto obliga al cliente de la API a calcular bien cuánto cobrarle al comprador antes de mandar la venta; el servidor no "ajusta" nada por su cuenta.

Paso 6 — Persistir: padre, después hijos

Recién acá, después de que *todas* las validaciones pasaron, se empieza a escribir en la base: primero `Sale` (la fila padre, para tener un `sale.id` real), después cada `SaleItem` y cada `SalePayment` con ese `sale_id` ya asignado. Este orden (padre primero, hijos después, usando el id generado) es obligatorio porque `sale_items.sale_id` es una clave foránea — no se puede insertar una línea de venta que apunte a una venta que todavía no existe.

> **El "gotcha" de copiar el id de vuelta:** fijate estas líneas:

```
item.sale_id = sale.id
created_item = await self._sale_item_repository.create(item)
item.id = created_item.id
```

> El repositorio `create(item)` **no modifica** el objeto `item` que le pasaste — internamente crea un modelo de SQLAlchemy nuevo, lo guarda, y devuelve un `SaleItem` de dominio *nuevo* (construido por `_to_domain`, sección 5.3) con el `id` que la base le asignó. Si no hicieras `item.id = created_item.id`, tu variable `item` original seguiría teniendo `id=None` — y como el objeto `SaleDetailRead` de la respuesta se arma a partir de la lista `items` original (no de `created_item`), la API devolvería líneas de venta sin id. Este patrón "crear, copiar el id de vuelta" se repite en Sale, Purchase, Promotion y Exchange, cada vez que se persisten hijos de un agregado.

9.4. La ruta y su helper de armado de respuesta

`app/routes/sale_route.py`

```
@router.post("", response_model=SaleDetailRead, status_code=status.HTTP_201_CREATED)
async def create_sale(
   payload: SaleCreate, service: SaleService = Depends(get_sale_service)
) -> SaleDetailRead:
   items = [
       SaleItem(stock_id=item.stock_id, quantity=item.quantity, unit_price=item.unit_price)
       for item in payload.items
   ]
   payments = [
       SalePayment(
           payment_method_id=payment.payment_method_id,
           amount_without_surcharge=payment.amount_without_surcharge,
           amount_with_surcharge=payment.amount_with_surcharge,
       )
```

```
        for payment in payload.payments
    ]
    sale = await service.create_sale(
        payload.branch_id,
        payload.discount,
        items,
        payments,
        discount_reason=payload.discount_reason,
    )
    return _to_detail_read(
        sale.id, sale.branch_id, sale.subtotal, sale.discount,
        sale.promotion_discount, sale.discount_reason, sale.total,
        items, payments,
    )
```

La ruta convierte cada `SaleItemInput`/`SalePaymentInput` (Pydantic, capa API) en `SaleItem`/`SalePayment` (dataclass, capa dominio) usando listas por comprensión. Después de llamar al service, arma la respuesta con `_to_detail_read` usando la **misma lista `items`** que le pasó al service — y funciona correctamente porque, como vimos arriba, el service le fue mutando `item.id`, `item.sale_id` e `item.promotion_id` a esos mismos objetos en memoria durante el Paso 4 y el Paso 6. `SaleItemRead.model_validate(item)` (dentro de `_to_detail_read`) convierte cada dataclass de dominio en el schema de salida gracias al `from_attributes=True` que ya vimos en la sección 5.4.

10. Purchase y Exchange: variaciones del mismo patrón

Ya viste el patrón completo de "agregado con padre e hijos, unit of work, copiar el id de vuelta" con Sale. Purchase y Exchange son el **mismo** patrón aplicado a otros dos flujos de negocio. En vez de repetir la explicación entera, esta sección se enfoca sólo en **qué es distinto**.

10.1. Purchase: el espejo de Sale

app/services/purchase_service.py

```
async def create_purchase(
   self,
   branch_id: int,
   supplier_name: str | None,
   items: list[PurchaseItem],
) -> Purchase:
   for item in items:
       stock = await self._stock_service.get_by_id(item.stock_id)
       if stock.branch_id != branch_id:
           raise BusinessRuleViolationError(
               f"Stock {item.stock_id} belongs to branch {stock.branch_id}, not {branch_id}"
           )

   subtotal = sum((item.unit_cost * item.quantity for item in items), Decimal("0"))
   total = subtotal

   purchase = await self._purchase_repository.create(
       Purchase(
           branch_id=branch_id, supplier_name=supplier_name, subtotal=subtotal, total=total
       )
   )

   for item in items:
       await self._stock_service.increase_quantity(item.stock_id, item.quantity)
       item.purchase_id = purchase.id
       created_item = await self._purchase_item_repository.create(item)
       item.id = created_item.id

   return purchase
```

Diferencias concretas contra `create_sale`:

- **Suma stock en vez de restarlo** (`increase_quantity` en vez de `decrease_quantity`): una compra ingresa mercadería, una venta la saca.
- **No tiene promociones ni pagos múltiples.** `supplier_name` es un campo de texto libre (no hay una entidad `Supplier` separada — una decisión de simplicidad: si en el futuro hiciera falta un historial de proveedores con más datos, ahí sí se justificaría una entidad propia).
- **Valida la sucursal del stock contra la sucursal de la compra, al principio, antes de tocar nada** (el bucle `for item in items: ... raise BusinessRuleViolationError` al inicio del método). Esto se agregó porque `PurchaseItem` identifica el stock a incrementar por `stock_id` (una fila concreta, que ya tiene su propia sucursal fijada desde que se creó), no por `branch_id` — sin esta validación, nada impedía mandar un `stock_id` de una sucursal distinta a la declarada en la compra y que el

sistema lo aceptara silenciosamente.

- **Es "append-only"**: `PurchaseRepository` (como `SaleRepository`) sólo tiene `list_all`/`get_by_id`/`create` — nunca `update` ni `delete`. La idea de negocio es que una compra o una venta ya concretada no se edita después; si hay un error, se corrige con otra operación (en el caso de ventas, un `Exchange`), no reescribiendo el historial.

10.2. Exchange: la operación más compuesta

`Exchange` modela tanto un cambio de producto como una devolución pura (una devolución es un cambio sin ítems nuevos). Tiene **tres** tipos de hijos en vez de dos: `ExchangeReturnItem` (lo que el cliente devuelve, repone stock), `ExchangeNewItem` (lo que se lleva a cambio, si corresponde, descuenta stock) y `ExchangePayment` (la diferencia a favor o en contra, con signo — positivo si el cliente paga de más, negativo si se le devuelve plata).

`app/services/exchange_service.py (fragmento clave)`

```
sold_quantity_by_stock: dict[int, int] = {}
for sale_item in await self._sale_item_repository.list_by_sale(sale_id):
   sold_quantity_by_stock[sale_item.stock_id] = (
       sold_quantity_by_stock.get(sale_item.stock_id, 0) + sale_item.quantity
   )

for returned_item in returned_items:
   sold_quantity = sold_quantity_by_stock.get(returned_item.stock_id)
   if sold_quantity is None:
       raise BusinessRuleViolationError(
           f"Stock {returned_item.stock_id} was not part of sale {sale_id}"
       )
   if returned_item.quantity > sold_quantity:
       raise BusinessRuleViolationError(
           f"Cannot return {returned_item.quantity} unit(s) of stock "
           f"{returned_item.stock_id}; only {sold_quantity} were sold"
       )

returned_total = sum((item.unit_price * item.quantity for item in returned_items),
Decimal("0"))
new_total = sum((item.unit_price * item.quantity for item in new_items), Decimal("0"))
balance = new_total - returned_total

paid_balance = sum((payment.amount for payment in payments), Decimal("0"))
if paid_balance != balance:
   raise BusinessRuleViolationError(
       f"Payments total ({paid_balance}) does not match the exchange balance ({balance})"
   )
```

Lo interesante acá: antes de aceptar una devolución, el service reconstruye `sold_quantity_by_stock` — cuánto se vendió de cada `stock_id` en la venta original — y valida que no se intente devolver más de lo que efectivamente se vendió (ni algo que nunca formó parte de esa venta). El `balance` es `new_total - returned_total`: si el cliente se lleva algo más caro de lo que devuelve, `balance` es positivo (tiene que pagar la diferencia); si se lleva algo más barato o no se lleva nada, es negativo (hay que devolverle plata). Los `ExchangePayment.amount` tienen que sumar exactamente ese balance, con signo.

> **Una limitación conocida, documentada a propósito en el código** (mirá el docstring de `ExchangeService` arriba de la clase): `sold_quantity_by_stock` se calcula sólo a partir de la venta

> original, sin descontar lo que ya se hubiera devuelto en cambios *anteriores* sobre esa misma venta. En teoría, alguien podría devolver las mismas unidades dos veces usando dos `Exchange` distintos contra la misma `Sale`. Se dejó así a propósito para no sumarle una consulta agregada (o un contador) que complicaría el código sin que, por ahora, sea un riesgo real para este proyecto. Documentar explícitamente una limitación conocida (en vez de ocultarla o "arreglarla mal" con un parche apurado) es una práctica de ingeniería tan válida como resolverla — siempre que quede registrada para revisarla si alguna vez importa.

11. Caso de estudio a fondo: Promotion + PromotionItem

Esta es la parte más compleja de todo el sistema, así que vale la pena repasar primero **el problema de negocio** antes de mirar el código. Se necesitaba resolver tres cosas:

1. Una promoción simple: "10% off en remeras".
2. Un combo: "15% off si comprás short y remera juntos" — la condición involucra varios productos a la vez, no uno solo.
3. Un regalo con descuentos distintos por producto: "llevando 1 remera, te regalamos 1 par de medias" — acá el descuento de la remera es 0% y el de las medias es 100%, y sólo se regala *un* par aunque el cliente lleve varios.

Y además: cuando varias promociones podrían aplicar a la vez sobre el mismo carrito, el sistema tiene que decidir **cuál** aplicar, priorizando la que le convenga más al cliente en pesos (no en porcentaje).

11.1. Por qué se separó en padre + hijos

La primera versión de `Promotion` tenía `product_id`, `category_id`, `discount_type` y `discount_value` directamente en la tabla — una promoción, un único producto o categoría, un único porcentaje. Esa forma no puede representar un combo (necesita *varios* productos con *distintos* porcentajes). La solución: separar en dos tablas, igual que Sale/SaleItem.

`app/domain/promotion.py`

```
from dataclasses import dataclass
from datetime import date
from enum import StrEnum


class DiscountType(StrEnum):
   PERCENTAGE = "percentage"
   FIXED_AMOUNT = "fixed_amount"


@dataclass(slots=True)
class Promotion:
   name: str
   start_date: date
   end_date: date
   active: bool = True
   id: int | None = None
```

`Promotion` quedó reducida a lo que es común a cualquier promoción, sin importar su forma: un nombre, un rango de fechas de vigencia, y si está activa. `DiscountType` es un `StrEnum` (enumeración cuyos valores son strings): sólo puede valer `"percentage"` o `"fixed_amount"`, nunca otra cosa — usar un enum en vez de un `str` suelto evita errores de tipeo ("percentaje" en vez de "percentage") que Python no detectaría hasta ejecutar el código.

`app/domain/promotion_item.py`

```
from dataclasses import dataclass
from decimal import Decimal

from app.domain.promotion import DiscountType


@dataclass(slots=True)
class PromotionItem:
   """min_quantity is the trigger threshold (how many of the target must
   be in the cart for this line to count as satisfied); max_quantity caps
   how many units actually get discounted once triggered (None = discount
   the full matched quantity in the cart). They're separate so a plain
   "10% off shirts" line (max_quantity=None) discounts every shirt bought,
   while a gift line like "buy 1 shirt, get 1 pair of socks free"
   (max_quantity=1) only discounts a single pair even if more are in the
   cart."""

   promotion_id: int | None
   discount_type: DiscountType
   discount_value: Decimal
   product_id: int | None = None
   category_id: int | None = None
   min_quantity: int = 1
   max_quantity: int | None = None
   id: int | None = None
```

Cada `PromotionItem` es una "línea" dentro de una promoción: apunta a exactamente un `product_id` **o** un `category_id` (nunca los dos, nunca ninguno — se valida en `_validate_items`, sección 11.3), con su propio tipo y valor de descuento. Una promoción con una sola línea es el caso simple ("10% off remeras"); con dos o más líneas es un combo o un regalo.

La distinción entre `min_quantity` y `max_quantity` es la parte más sutil de todo el diseño, y vale la pena que la entiendas bien porque no es obvia a primera vista:

- `min_quantity` responde "¿cuánto hace falta en el carrito para que esta línea *se active*?" Es la condición de entrada.

- `max_quantity` responde "de lo que hay en el carrito, ¿cuánto se descuenta como máximo?" Es el techo del beneficio. `None` significa "sin techo, descontá todo lo que haya".

Para "10% off remeras" (una sola línea, `min_quantity=1`, `max_quantity=None`): alcanza con tener 1 remera para activar la promo, y se descuentan *todas* las remeras del carrito, sean 1 o 10. Para el regalo de medias (`min_quantity=1`, `max_quantity=1` en la línea de medias): hace falta al menos 1 remera y 1 par de medias para activarse, pero sólo se regala *un* par, aunque el carrito tenga 3.

11.2. El modelo y el repositorio de PromotionItem

El modelo SQLAlchemy (`app/infrastructure/db/models/promotion_item_model.py`) sigue el mismo patrón que ya viste: una tabla `promotion_items` con `promotion_id` como clave foránea obligatoria, y el resto de los campos reflejando el dominio 1:1. El repositorio también sigue el patrón conocido (ABC + `SqlAlchemy*`), con un método extra:

```
async def delete_by_promotion(self, promotion_id: int) -> None:
    await self._session.execute(
        delete(PromotionItemModel).where(PromotionItemModel.promotion_id == promotion_id)
    )
    await self._session.flush()
```

Esto lo usa `PromotionService.update` (a continuación) para implementar "reemplazo total" de las líneas de una promoción al editarla: en vez de intentar calcular qué líneas cambiaron, cuáles se borraron y cuáles son nuevas (lógica de "diff" bastante más compleja de escribir bien), es más simple y menos propenso a errores borrar todas las líneas existentes y volver a crear las que llegaron en la petición de `update`.

11.3. El servicio: creación, validación, y el algoritmo de selección

`app/services/promotion_service.py (creación y validación)`

```
async def create(self, promotion: Promotion, items: list[PromotionItem]) -> Promotion:
    await self._validate_items(items)
    created = await super().create(promotion)
    for item in items:
        item.promotion_id = created.id
        new_item = await self._promotion_item_repository.create(item)
        item.id = new_item.id
    return created

async def _validate_items(self, items: list[PromotionItem]) -> None:
    if not items:
        raise BusinessRuleViolationError("Promotion must have at least one item")
    for item in items:
        if (item.product_id is None) == (item.category_id is None):
            raise BusinessRuleViolationError(
                "Promotion item must target exactly one of product_id or category_id"
            )
        if item.product_id is not None:
            if await self._product_repository.get_by_id(item.product_id) is None:
                raise EntityNotFoundError("Product", item.product_id)
        if item.category_id is not None:
            if await self._category_repository.get_by_id(item.category_id) is None:
                raise EntityNotFoundError("Category", item.category_id)
```

`create` sigue exactamente el patrón "padre, después hijos, copiar id" de las secciones 9.3/10. Lo interesante es `_validate_items`: fijate la expresión `(item.product_id is None) == (item.category_id is None)`. Es un truco compacto para "exactamente uno de los dos": si ambos son `None`, la comparación da `True == True` → `True` (error, ninguno de los dos). Si ambos tienen valor, da `False == False` → `True` (error, los dos a la vez). Sólo si uno es `None` y el otro no, da `True == False` → `False` (no hay error). Es la misma lógica que un XOR (o exclusivo) aplicado a "está en `None`".

`PromotionService` también hereda de `CrudService[Promotion]` (igual que `CategoryService`, sección 5.5), pero **sobrescribe** `create` con una firma distinta (recibe `items` además de `promotion`) — Python lo permite sin problema, aunque técnicamente "rompe" la firma exacta del método heredado. Es una decisión consciente: reusar el `get_by_id`/`list_all`/`delete` heredados (que sí funcionan igual que en cualquier otra entidad), pero necesitar más información para `create`/`update` porque acá el "objeto completo" es padre + hijos, no sólo el padre.

El algoritmo de selección, paso a paso

Este es el método que `SaleService.create_sale` llama (sección 9.3, Paso 3). Recibe el "resumen del carrito" (los diccionarios `cart_quantities`/`cart_categories`/ `cart_unit_prices`) y una fecha, y devuelve la lista de descuentos ganadores.

`app/services/promotion_service.py (el algoritmo completo)`

```
async def select_winning_applications(
   self,
   cart_quantities: dict[int, int],
   cart_categories: dict[int, int],
   cart_unit_prices: dict[int, Decimal],
   on_date: date,
) -> list[PromotionApplication]:
   candidates = await self._collect_satisfied_candidates(
       cart_quantities, cart_categories, cart_unit_prices, on_date
   )
   if not candidates:
       return []

   union_find = _UnionFind([candidate.promotion_id for candidate in candidates])
   for i in range(len(candidates)):
       for j in range(i + 1, len(candidates)):
           if candidates[i].matched_products & candidates[j].matched_products:
               union_find.union(candidates[i].promotion_id, candidates[j].promotion_id)

   groups: dict[int, list[_Candidate]] = {}
   for candidate in candidates:
       root = union_find.find(candidate.promotion_id)
       groups.setdefault(root, []).append(candidate)

   applications: list[PromotionApplication] = []
   for group in groups.values():
       winner = max(group, key=lambda c: (c.total_discount, -c.promotion_id))
       applications.extend(winner.applications)

   applications.sort(key=lambda a: (a.promotion_id, a.product_id))
   return applications
```

El método se apoya en dos ayudantes: `_collect_satisfied_candidates` (que arma la lista de promociones que *podrían* aplicar) y `_UnionFind` (una estructura de datos clásica para agrupar cosas relacionadas). Vamos por partes.

Paso A — ¿Qué promociones están "satisfechas" por el carrito?

```
async def _collect_satisfied_candidates(
   self, cart_quantities, cart_categories, cart_unit_prices, on_date
) -> list[_Candidate]:
   candidates: list[_Candidate] = []
   for promotion in await self._repository.list_active(on_date):
       lines = await self._promotion_item_repository.list_by_promotion(promotion.id)
       if not lines:
           continue

       remaining = dict(cart_quantities)
       matched: list[tuple[int, PromotionItem, int]] = []
       satisfied = True
       for line in lines:
```

```
            target_product_id = self._resolve_line_target(line, remaining, cart_categories)
            available = remaining.get(target_product_id, 0) if target_product_id else 0
            if target_product_id is None or available < line.min_quantity:
                satisfied = False
                break
            if line.max_quantity is not None:
                effective_quantity = min(line.max_quantity, available)
            else:
                effective_quantity = available
            remaining[target_product_id] -= effective_quantity
            matched.append((target_product_id, line, effective_quantity))

        if not satisfied:
            continue
        # ... (calcula total_discount por línea, arma el _Candidate; ver más abajo)
```

Por cada promoción activa en la fecha (`list_active(on_date)` ya filtra por `active=True` y rango de fechas), se recorren sus líneas **en orden**. Se mantiene un diccionario `remaining`, copia de `cart_quantities`: representa "cuánto de cada producto todavía no fue reclamado por ninguna línea de *esta* promoción". Es una copia nueva por cada promoción evaluada — así, evaluar la Promoción B no contamina lo que ya se "gastó" evaluando la Promoción A (son candidatas independientes hasta que se decide cuál gana).

Para cada línea: se resuelve a qué producto apunta (`_resolve_line_target`, ver abajo); si no hay suficiente cantidad disponible (`available < line.min_quantity`), la promoción entera queda descartada (`satisfied = False; break` — no hace falta seguir revisando las demás líneas). Si sí alcanza, se calcula `effective_quantity` (aplicando el techo de `max_quantity` si existe, sección 11.1) y se **resta** de `remaining[target_product_id]`. Esa resta es la clave para evitar que dos líneas de la misma promoción reclamen las mismas unidades dos veces (por ejemplo, si una promoción tuviera dos líneas apuntando al mismo producto).

```
@staticmethod
def _resolve_line_target(line, remaining, cart_categories) -> int | None:
   if line.product_id is not None:
       return line.product_id
   for product_id, category_id in cart_categories.items():
       if category_id != line.category_id:
           continue
       if remaining.get(product_id, 0) >= line.min_quantity:
           return product_id
   return None
```

Si la línea apunta a un `product_id` puntual, ese es el objetivo, sin más vuelta. Si apunta a una `category_id`, se recorre `cart_categories` (en el orden en que los productos entraron al diccionario, que es el orden en que aparecieron en el carrito) buscando el **primer** producto de esa categoría que todavía tenga suficiente cantidad disponible. Es una simplificación consciente: si el carrito tuviera dos productos distintos de la misma categoría, la línea sólo "elige" uno, no reparte el descuento entre ambos.

Paso B — Calcular cuánto descuenta cada candidata, en pesos

```
applications: list[PromotionApplication] = []
total_discount = Decimal("0")
for product_id, line, effective_quantity in matched:
   unit_price = cart_unit_prices.get(product_id, Decimal("0"))
   discount = compute_line_discount(
```

```
            line.discount_type, line.discount_value, unit_price, effective_quantity
        )
        total_discount += discount
        applications.append(
            PromotionApplication(
                promotion_id=promotion.id, product_id=product_id,
                quantity=effective_quantity, discount_amount=discount,
            )
        )

    candidates.append(
        _Candidate(
            promotion_id=promotion.id,
            applications=applications,
            matched_products={product_id for product_id, _, _ in matched},
            total_discount=total_discount,
        )
    )
```

Por cada línea que sí se activó, se calcula el descuento en pesos con la función compartida `compute_line_discount` (porcentaje sobre el subtotal de esa cantidad, o monto fijo tapado para no superar el subtotal — redondeado a centavos con `ROUND_HALF_UP`, el redondeo "de toda la vida": 0.5 siempre redondea para arriba). Se suman todas las líneas de la promoción en `total_discount`: **esto es lo que después decide quién gana**, no el porcentaje.

Paso C — Agrupar candidatas que compiten entre sí (union-find)

Acá está la parte conceptualmente más nueva: `_UnionFind`. Es una estructura de datos clásica para resolver el problema "agrupar elementos en conjuntos, donde sé que ciertos pares están relacionados, pero no sé de antemano cuáles terminan en el mismo grupo". Se usa muchísimo en problemas de grafos y redes.

```
class _UnionFind:
    def __init__(self, ids: list[int]) -> None:
        self._parent = {i: i for i in ids}

    def find(self, x: int) -> int:
        while self._parent[x] != x:
            self._parent[x] = self._parent[self._parent[x]]
            x = self._parent[x]
        return x

    def union(self, a: int, b: int) -> None:
        root_a, root_b = self.find(a), self.find(b)
        if root_a != root_b:
            self._parent[root_b] = root_a
```

La idea: cada elemento arranca siendo su propio "representante" (`self._parent = {i: i for i in ids}`, cada uno apunta a sí mismo). `union(a, b)` junta los grupos de `a` y `b` haciendo que el representante de uno apunte al representante del otro. `find(x)` sigue la cadena de punteros hasta encontrar el representante final del grupo de `x` (la línea `self._parent[x] = self._parent[self._parent[x]]` "acorta" el camino mientras busca, una optimización llamada *path compression* que evita que la estructura se vuelva lenta con el tiempo). Dos elementos están en el mismo grupo si y sólo si `find(a) == find(b)`.

El algoritmo compara **cada par** de candidatas (`for i ... for j in range(i+1, ...)`, todas las combinaciones sin repetir) y las une si `matched_products` (el conjunto de productos que cada una tocó) **se superpone** (`&` es la intersección de conjuntos en Python; si el resultado no está vacío, hay al menos un producto en común). Así, si la Promoción A comparte un producto con la B, y la B comparte otro producto con la C (aunque A y C no compartan nada directamente entre sí), las tres terminan en el mismo grupo por transitividad — exactamente el comportamiento que hace falta: "todas las promociones que de alguna forma se pisan sobre el carrito, compiten entre sí como un solo bloque".

Paso D — Elegir la ganadora de cada grupo

```
groups: dict[int, list[_Candidate]] = {}
for candidate in candidates:
   root = union_find.find(candidate.promotion_id)
   groups.setdefault(root, []).append(candidate)

applications: list[PromotionApplication] = []
for group in groups.values():
   winner = max(group, key=lambda c: (c.total_discount, -c.promotion_id))
   applications.extend(winner.applications)
```

`groups` junta las candidatas por su representante de union-find. Dentro de cada grupo, `max(group, key=lambda c: (c.total_discount, -c.promotion_id))` elige la de mayor `total_discount`; en caso de empate exacto, gana la de `promotion.id` más chico (`-c.promotion_id` invierte el orden: como `max` busca el valor más alto, y queremos el id más bajo en un empate, se compara el negativo). Grupos distintos (sin ningún producto en común) ganan **todos**, en simultáneo — no compiten entre sí porque no se tocan.

11.4. El ejemplo completo, verificado en el sistema real

Repasemos el caso que motivó todo este diseño, con números reales (probado directamente contra la API corriendo, no a mano): remera $30.000, short $20.000, medias $15.000. Tres promociones activas compitiendo:

Promoción	Productos	%	Descuento en $
A	Remera sola	10%	$3.000
B	Remera + Short	15%	$7.500
C	Remera + Short + Medias	12%	$7.800

Alguien compra las tres prendas juntas. Las tres promociones comparten al menos la remera — union-find las agrupa a las tres en un único grupo. Dentro de ese grupo, gana la Promoción C, con $7.800 de descuento, **a pesar de tener el menor porcentaje de las tres** (12% contra 15% de la B), porque en pesos es la que más beneficia al cliente. Total a pagar: $65.000 – $7.800 = **$57.200**. Así respondió el sistema real al probarlo:

```
{
 "subtotal": "65000",
 "promotion_discount": "7800.00",
 "total": "57200.00",
```

```
  "items": [
    {"stock_id": 6, "unit_price": "30000", "promotion_id": 8},
    {"stock_id": 7, "unit_price": "20000", "promotion_id": 8},
    {"stock_id": 8, "unit_price": "15000", "promotion_id": 8}
  ]
}
```

Las tres líneas quedaron marcadas con `promotion_id: 8` (la Promoción C), confirmando que ganó ella y cubrió los tres productos.

11.5. Rutas y schemas: el mismo patrón compuesto que Sale

`app/schemas/promotion_schema.py` sigue el mismo patrón compuesto que `SaleCreate` (sección 9.2): `PromotionItemInput` (lo que manda el cliente por cada línea), `PromotionCreate`/`PromotionUpdate` con `items: list[PromotionItemInput]`, y `PromotionRead`/`PromotionDetailRead` para la respuesta. La ruta arma `Promotion` + `list[PromotionItem]` a partir del payload y llama a `service.create(promotion, items)`, igual que `sale_route.py` lo hace con `SaleCreate`. No hace falta repetir ese código acá — es exactamente la misma forma que ya viste dos veces.

12. Testing: cómo se prueba todo esto

El proyecto usa `pytest` (el framework de testing estándar de facto en Python) junto con `pytest-asyncio` (que le enseña a pytest a ejecutar funciones de test `async def`, algo que pytest no sabe hacer solo). Los tests corren contra una base SQLite **en memoria**, no contra Postgres — como vimos en la sección 3.5, esto es posible porque el código de acceso a datos nunca usa nada específico de un motor en particular.

12.1. Las fixtures compartidas

`tests/conftest.py`

```
@pytest_asyncio.fixture
async def engine() -> AsyncGenerator[AsyncEngine, None]:
   test_engine = create_async_engine(
       "sqlite+aiosqlite:///:memory:",
       poolclass=StaticPool,
       connect_args={"check_same_thread": False},
   )
   async with test_engine.begin() as conn:
       await conn.run_sync(Base.metadata.create_all)

   yield test_engine

   await test_engine.dispose()


@pytest_asyncio.fixture
def session_factory(engine: AsyncEngine) -> async_sessionmaker[AsyncSession]:
   return async_sessionmaker(bind=engine, expire_on_commit=False)


@pytest_asyncio.fixture
async def db_session(
   session_factory: async_sessionmaker[AsyncSession],
) -> AsyncGenerator[AsyncSession, None]:
   async with session_factory() as session:
       yield session


@pytest_asyncio.fixture
async def client(
   session_factory: async_sessionmaker[AsyncSession],
) -> AsyncGenerator[AsyncClient, None]:
   async def override_get_db() -> AsyncGenerator[AsyncSession, None]:
       async with session_factory() as session:
           try:
               yield session
               await session.commit()
           except Exception:
               await session.rollback()
               raise

   app.dependency_overrides[get_db] = override_get_db

   transport = ASGITransport(app=app)
   async with AsyncClient(transport=transport, base_url="http://test") as async_client:
```

```
        yield async_client

    app.dependency_overrides.clear()
```

Un *fixture* de pytest es una función que prepara algo que un test necesita (una conexión, un objeto ya armado) y se lo "inyecta" al test simplemente nombrándolo como parámetro. Acá hay una cadena de cuatro fixtures, cada una construida sobre la anterior:

- `engine`: crea un motor de SQLite **en memoria** (`sqlite+aiosqlite:///:memory:`, no un archivo real) y le crea todas las tablas de una (`Base.metadata.create_all` — acá se ve por qué importa que todos los modelos se hayan importado antes, sección 6.1: si falta un `import`, esa tabla no se crea y los tests que la necesiten van a fallar). `StaticPool` asegura que todas las conexiones dentro de un mismo test compartan la misma base en memoria (SQLite en memoria es "por conexión" por default, así que sin esto cada consulta vería una base distinta y vacía).
- `session_factory`: una fábrica de sesiones sobre ese motor de test.
- `db_session`: una sesión concreta, para tests que llaman directamente a servicios (sin pasar por HTTP).
- `client`: un cliente HTTP falso, para tests que llaman a las *rutas* reales. `app.dependency_overrides[get_db] = override_get_db` es la pieza clave: le dice a FastAPI "cuando alguien pida `Depends(get_db)`, usá esta otra función en su lugar" — así toda la app, corriendo de verdad, termina usando la base de test sin que ningún código de producción se entere. `ASGITransport` conecta el cliente HTTP directamente a la app en memoria, sin abrir un socket de red real, así que las pruebas de rutas corren tan rápido como las pruebas de servicios.

12.2. Dos niveles de test: servicio y ruta

El proyecto prueba la lógica de negocio en dos niveles:

- **Tests de servicio** (`test_sale_service.py`, `test_promotion_service.py`, etc.): usan la fixture `db_session` y llaman *directamente* a los métodos de un service, armando a mano sus repositorios. Son rápidos y prueban la lógica de negocio en aislamiento, sin pasar por HTTP ni por la validación de Pydantic.
- **Tests de ruta** (`test_promotion_routes.py`, `test_purchase_routes.py`, etc.): usan la fixture `client` y hacen peticiones HTTP reales (`await client.post("/products", json={...})`), verificando también la validación de entrada y los códigos de estado.

Ejemplo de test de ruta completo, con el patrón típico "armar dependencias por HTTP, después probar el caso real":

tests/test_product_routes.py

```
async def test_create_product_with_missing_category_returns_404(client: AsyncClient) ->
None:
    brand_resp = await client.post("/brands", json={"name": "Adidas"})
    brand_id = brand_resp.json()["id"]

    resp = await client.post(
        "/products",
        json={
```

```
            "name": "Zapatilla",
            "category_id": 999,
            "brand_id": brand_id,
            "cost": "10.00",
            "price": "20.00",
        },
    )
    assert resp.status_code == 404
```

Este test verifica de punta a punta que `ProductService._ensure_references_exist` (sección 7) realmente devuelve un 404 cuando `category_id` no existe, pasando por toda la cadena real: ruta → schema → servicio → excepción → handler HTTP.

12.3. Un patrón de test de servicio, para que veas la forma

Los tests de servicio suelen tener esta estructura: una función `_seed_...` que prepara los datos previos necesarios (crear categoría, marca, producto, stock — usando los repositorios directamente), una función `_build_..._service` que arma el service bajo prueba con sus dependencias reales apuntando a la sesión de test, y el test en sí, que llama al método bajo prueba y hace `assert` sobre el resultado:

```
async def test_create_purchase_rejects_stock_from_a_different_branch(
    db_session: AsyncSession,
) -> None:
    stock = await _seed_stock(db_session, quantity=5)
    other_branch = await SqlAlchemyBranchRepository(db_session).create(Branch(name="Sucursal
2"))
    service = _build_purchase_service(db_session)

    with pytest.raises(BusinessRuleViolationError):
        await service.create_purchase(
            branch_id=other_branch.id,
            supplier_name=None,
            items=[PurchaseItem(stock_id=stock.id, quantity=3, unit_cost=Decimal("10.00"))],
        )

    unchanged_stock = await SqlAlchemyStockRepository(db_session).get_by_id(stock.id)
    assert unchanged_stock is not None
    assert unchanged_stock.quantity == 5
```

`with pytest.raises(BusinessRuleViolationError):` es la forma de pytest de decir "espero que el código de adentro de este bloque lance esta excepción; si no la lanza, el test falla". Fijate que el test también verifica que la cantidad de stock **no haya cambiado** después del error — confirma que la validación corta la operación *antes* de mutar nada, no a mitad de camino.

13. Migraciones con Alembic

Ya vimos qué es Alembic conceptualmente (sección 3.4). Veamos ahora cómo se usa en la práctica y qué errores reales aparecieron al aplicarlo en este proyecto — son buenos ejemplos de "cosas que hay que revisar siempre", no casos aislados.

13.1. El flujo normal de trabajo

1. Cambiás un modelo de SQLAlchemy (agregás una columna, una tabla nueva, etc.).
2. Corrés `uv run alembic revision --autogenerate -m "descripción del cambio"`. Alembic compara `Base.metadata` (lo que *debería* existir, según tus modelos actuales) contra el estado real de la base de datos, y genera un archivo Python nuevo en `alembic/versions/` con el `upgrade()`/`downgrade()` que hacen falta para llegar de un estado al otro.
3. **Revisás ese archivo a mano** — nunca se aplica un autogenerate a ciegas.
4. Corrés `uv run alembic upgrade head` para aplicarlo contra la base real.

Cada migración declara `down_revision`: el id de la migración anterior. Así forman una cadena lineal (como una lista enlazada) que Alembic recorre para saber en qué orden aplicarlas. El proyecto tiene, en este momento, cuatro migraciones encadenadas: esquema inicial → agrega Purchase/Promotion/Exchange → Stock opcional + discount_reason + promotion_id → PromotionItem (combos).

13.2. Por qué "revisar a mano" no es opcional: dos errores reales

Caso 1: NOT NULL sobre una columna con filas existentes

Al agregar `Sale.promotion_discount` como obligatoria (`nullable=False`) en una tabla que ya tenía ventas cargadas, la migración autogenerada intentaba:

```
ALTER TABLE sales ADD COLUMN promotion_discount NUMERIC(12, 2) NOT NULL
```

Esto falla en Postgres: las filas que ya existen no tienen ningún valor para la columna nueva, y "obligatoria" (`NOT NULL`) sin un valor por defecto es una contradicción. La solución fue agregar un `server_default='0'` temporal (para que las filas viejas se completen con `0` automáticamente) y sacarlo enseguida después con un `op.alter_column(..., server_default=None)`, para que el modelo de Python siga sin tener un default a nivel de base — sólo se usó para completar las filas viejas al momento de migrar:

```
op.add_column('sales', sa.Column('promotion_discount', sa.Numeric(12, 2),
                                 nullable=False, server_default='0'))
op.alter_column('sales', 'promotion_discount', server_default=None)
```

Regla general: cualquier columna nueva marcada como obligatoria, en una tabla que ya puede tener filas, necesita o un `server_default` temporal como éste, o ser nullable, o un paso previo de `UPDATE` que la complete antes del `ALTER TABLE ... NOT NULL`.

Caso 2: reusar un tipo ENUM ya existente

Al mover `discount_type` de `promotions` a la nueva tabla `promotion_items`, la migración autogenerada intentaba crear el mismo tipo `ENUM` de Postgres (`discounttype`) dos veces — una vez ya existía (de la columna vieja, que se borra en la misma migración, pero recién *después*), y Postgres rechazó la segunda creación:

```
sqlalchemy.exc.ProgrammingError: DuplicateObjectError: type "discounttype" already exists
```

La solución: indicarle explícitamente a SQLAlchemy que no intente crear el tipo de nuevo, con `create_type=False`:

```
sa.Column('discount_type',
   postgresql.ENUM('PERCENTAGE', 'FIXED_AMOUNT', name='discounttype', create_type=False),
   nullable=False)
```

Regla general: cuando una migración crea una columna nueva que reutiliza el nombre de un `ENUM` ya creado en la base (por otra tabla, o por una columna que se está por borrar en esa misma migración), agregale `create_type=False` para que Postgres no intente duplicarlo.

> Ninguno de estos dos errores es un bug del proyecto: son parte normal de trabajar con migraciones autogeneradas. Alembic es muy bueno detectando *qué* cambió entre tus modelos y la base real, pero no siempre sabe *cómo* aplicarlo de forma segura contra datos que ya existen. Revisar cada migración generada, pensando "¿esto puede fallar contra una base con datos reales?", es parte del trabajo, no un paso opcional.

14. Convenciones y buenas prácticas: checklist

Un resumen condensado para consultar rápido cuando tengas que agregar o tocar algo. Si algo de esta lista no te resulta obvio, es una señal de volver a la sección correspondiente de esta guía.

Situación	Qué hacer
Agregar una entidad de catálogo simple (sin reglas de negocio propias)	Copiar los cinco archivos de Category (dominio, modelo, repositorio, schema, service, ruta), renombrar, ajustar campos. El service hereda de `CrudService` sin agregar nada más (sección 5).
La entidad nueva depende de otra por clave foránea	Inyectar el repositorio de la entidad relacionada en el constructor del service, sobrescribir `create`/`update` llamando primero a una validación propia y después a `super().create()`/`super().update()` (patrón de Product y Stock, secciones 7 y 8).
La entidad nueva es un agregado (padre + hijos, ej. "Orden con líneas")	No heredar de `CrudService`. Escribir un service a medida con un método de creación que: valida todo primero, persiste el padre, después cada hijo copiando el `.id` de vuelta al objeto original (patrón de Sale/Purchase/Promotion/Exchange, secciones 9-11).
Agregar un campo a un modelo existente	Tocar en orden: dominio (dataclass), modelo SQLAlchemy, repositorio (`create`/`update`/`_to_domain`), schema Pydantic (Create y/o Read según si el cliente lo manda o sólo lo recibe), service si hace falta lógica nueva, ruta si cambia la firma. Generar migración con autogenerate y **revisarla a mano** (sección 13).
Un campo nuevo es "calculado por el servidor" (no lo manda el cliente)	No lo agregues al schema `*Create`/`*Input`, sólo al `*Read`/`*Output` (ejemplo: `SaleItem.promotion_id`, sección 9.2).
Manejo de errores	Nunca `HTTPException` en un service o repositorio. Lanzar `EntityNotFoundError`/`InsufficientStockError`/`BusinessRule` (o agregar una nueva si ninguna encaja, más su handler en `exception_handlers.py`, sección 6.5).
Nuevo archivo de modelo SQLAlchemy	Agregarlo al import de `alembic/env.py` Y de `tests/conftest.py`, en orden alfabético (sección 6.1) — si te olvidás de uno, esa tabla no se crea en las migraciones o en los tests.
Dinero (precios, costos, montos)	Siempre `Decimal`, nunca `float` (sección 7).

Testear un service nuevo	Test de servicio con `db_session`, armando el service a mano con sus repositorios reales sobre esa sesión (sección 12.3). Agregar también al menos un test de ruta si agrega endpoints nuevos.
¿Dónde vive la lógica de "wiring" (armar un service con sus dependencias)?	Al final de cada módulo de `app/services/`, en una función `get_<entidad>_service(session = Depends(get_db))`. No hay un archivo central de inyección de dependencias (sección 5.5).

15. Próximos pasos sugeridos

El sistema actual tiene, a propósito, algunas limitaciones documentadas (en `CLAUDE.md` del proyecto) que quedaron afuera del alcance hasta ahora. Son buenos ejercicios para practicar todo lo que viste en esta guía, de menor a mayor dificultad:

Ejercicio 1 (fácil): motivo en el descuento de Purchase

`Sale` ya tiene `discount_reason` (sección 9.1). `Purchase` no tiene un campo equivalente para anotar, por ejemplo, por qué se acordó un precio distinto con el proveedor. Practicá el flujo completo: dominio → modelo → repositorio → schema → service → ruta → migración → test.

Ejercicio 2 (intermedio): tope de usos por promoción

Hoy una `Promotion` se puede aplicar indefinidamente mientras esté activa y dentro de fecha. Agregá un campo (por ejemplo `max_uses: int | None` en `Promotion`, y un contador de usos, o una consulta que cuente cuántas veces ya se usó vía `SaleItem.promotion_id`) para que deje de aplicarse una vez alcanzado el límite. Vas a tener que decidir: ¿se cuenta por venta, o por línea de venta?

Ejercicio 3 (intermedio): transferencia de stock entre sucursales

`StockService` sólo tiene `increase_quantity`/`decrease_quantity` sueltos. Agregá un método `transfer(origin_stock_id, destination_stock_id, amount)` que haga ambas operaciones de forma atómica (aprovechando que comparten la misma sesión, sección 6.2), validando que el origen tenga stock suficiente antes de mover nada.

Ejercicio 4 (avanzado): promociones por cantidad ("3x2")

El diseño actual de `min_quantity`/`max_quantity` (sección 11.1) no repite una línea varias veces: comprar 6 unidades de un producto con una promo "3x2" hoy sólo dispara la línea una vez, no dos. Pensá cómo extenderías `PromotionItem` y el algoritmo de `_collect_satisfied_candidates` para soportar múltiplos.

Ejercicio 5 (avanzado): Exchange y promociones

Como vimos en la sección 10.2, `ExchangeService` calcula el valor de lo devuelto usando `unit_price * quantity`, sin considerar si esa línea original tenía un `promotion_id` aplicado. Investigá qué pasaría hoy si alguien devuelve un producto que se vendió con descuento, y diseñá cómo el sistema debería reflejar (o revertir proporcionalmente) ese descuento en la devolución.

> **Consejo final:** antes de escribir código para cualquiera de estos ejercicios, escribí primero el test que esperás que pase (aunque el código todavía no exista) — te obliga a pensar el comportamiento exacto que querés lograr antes de perderte en el "cómo". Es el mismo enfoque que se usó para construir el algoritmo de combos de la sección 11: primero se acordó, con ejemplos numéricos concretos, qué resultado se esperaba, y recién después se escribió el código.